Hopfield Digit Recognition Analysis

INTRODUCTION

Hopfield Network

The Hopfield Network is a type of recurrent neural network that was introduced by John Hopfield in 1982. It is used as a model for associative memory, pattern recognition, and solving some optimization problems.

This network is made of a group of neurons. Each neuron can have only two possible states which are usually represented by **+1** and **−1**. The main idea of a Hopfield network is that it can store some patterns and later retrieve them, even if the input pattern is noisy or incomplete.

The network works by trying to minimize an energy function. During this process, the network updates the neuron states until it reaches a stable condition. These stable states represent the patterns that were stored in the network. Because of this behavior, Hopfield networks show attractor dynamics, which means the system eventually converges to one of the stored patterns.

Network Architecture:

Fully Connected :A single-layer network where every neuron is connected to every other neuron, excluding self-connections. (w_{ii}= 0).

Symmetric Weights: The connection weight from neuron i to neuron j is the exact same as from j to i (w_{ij}= w_{ji}).

Bipolar States: Each neuron can take only two possible states, usually represented as $v_i \in \{-1, +1\}$. The collection of all neuron states represents the overall state of the network.

Training (Hebbian Learning) :

Training a Hopfield network is different from many modern neural networks because it does not use gradient descent or backpropagation. Instead, the network stores patterns using the **Hebbian learning rule**, which is based on the correlation between neuron activations. If we want to store Ppatterns, the weights of the network are computed using the following formula:

$$w_{ij} = \frac{1}{N}\sum_{p=1}^{P} x_i^{(p)} x_j^{(P)}, \qquad i \neq j$$

and

$$w_{ii} = 0$$

where $N$is the number of neurons and $x^{(p)}$represents the stored patterns. The diagonal elements of the weight matrix are set to zero.

State Update Rule:

During the recall phase, the neurons update their states according to an activation rule. In the general form used in the lecture notes, the update rule is defined as:

$$v_i^{k+1} = \text{sgn}\left(\sum_{j=1}^{n} w_{ij} v_j^k + i_i - T_i\right)$$

where w_{ij}represents the connection weights, $i_i$is the external input, and $T_i$is the threshold of the neuron.

The sign function is defined as:

$$sgn(x) = \begin{cases} +1 & x \geq 0 \\ -1 & x < 0 \end{cases}$$

Update Strategies:

There are two main ways to update neuron states.

- **Asynchronous**: Only one unit is updated at a time. This unit can be picked at random, or a pre-defined order can be imposed from the very beginning.
- **Synchronous:** All neurons evaluate their inputs and update their states at the exact same time during a single step.

Energy and Attractors:

One important characteristic of Hopfield networks is the existence of an **energy function** that describes the state of the system. The network always evolves toward states with lower energy. The energy of the system can be expressed as:

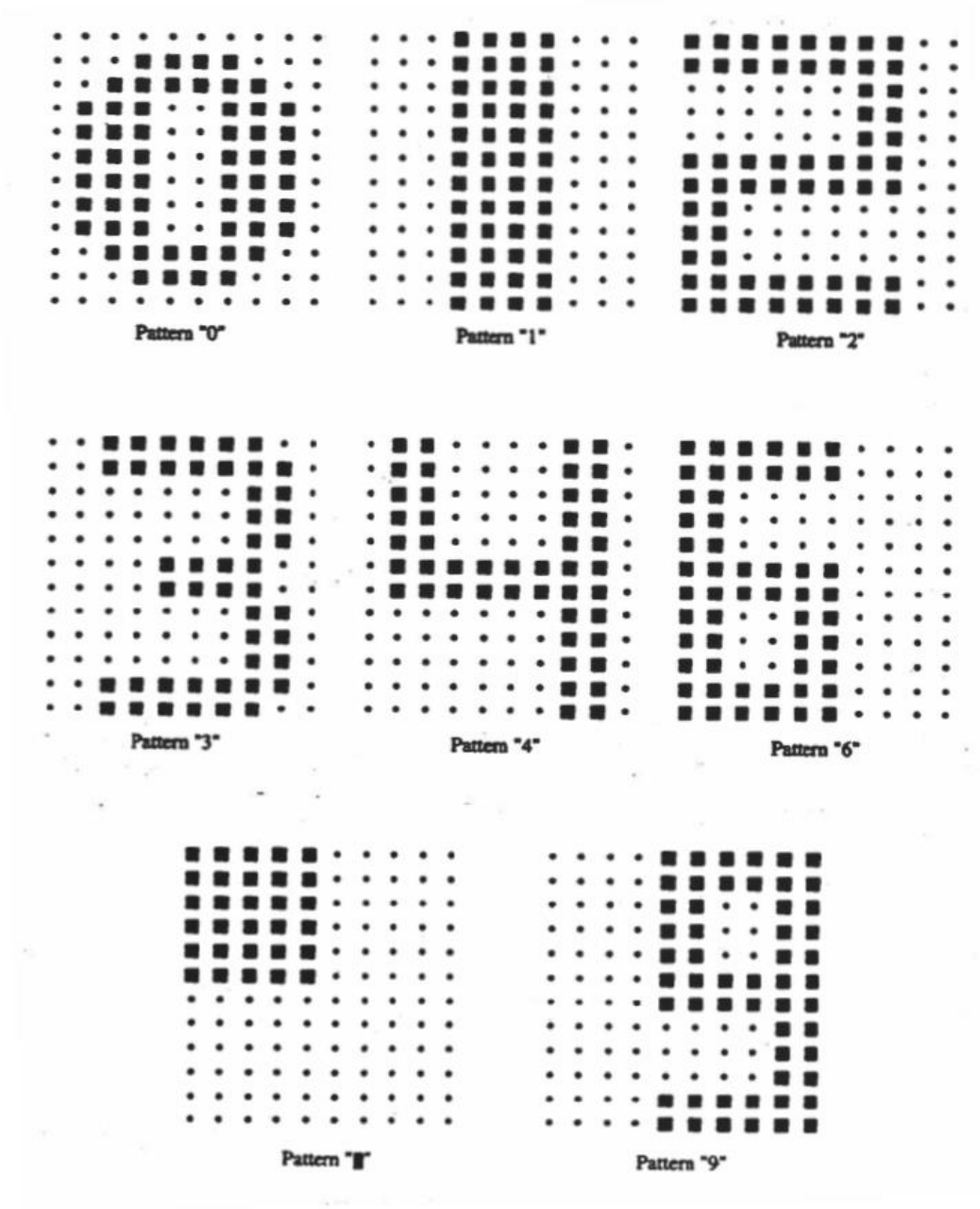
$$E = -\frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} w_{ij} v_i v_j - \sum_{i=1}^{n} i_i v_i + \sum_{i=1}^{n} T_i v_i$$

During the update process, the energy of the network decreases or remains constant, which guarantees that the system eventually reaches a stable state. These stable states are known as **attractors**, and they usually correspond to the patterns stored in the network.

PROBLEM

Design a Hopfield network to recognize the following patterns as an associator.

First, test the performance of the network in the recall mode using "clean" patterns. Next, test the performance of the net by corrupting the patterns with 25% noise, i.e. by randomly reversing 25 of pixels. Implement both synchronous and asynchronous learning procedures and comment on the performance Can all the patterns be recalled correctly?



Based on the document, here are the exact tasks i completed for this project:

- **Design the Network:** I designed a Hopfield network to act as an associator that recognized seven specific digit patterns: "0", "1", "2", "3", "4", "6", "9" , and ‘.’.
- **Test with Clean Patterns:** I tested the performance of the network in recall mode using the original, uncorrupted ("clean") patterns.
- **Test with Noisy Patterns:** I test the network's performance by corrupting the patterns with noise. Specifically, I randomly reversed 10 and another time 25 of the pixels in the input images.
- **Implement Two Update Methods:** I implemented both synchronous and asynchronous procedures.
- **Evaluate and Comment:** Finally, I commented on the overall performance of the network.

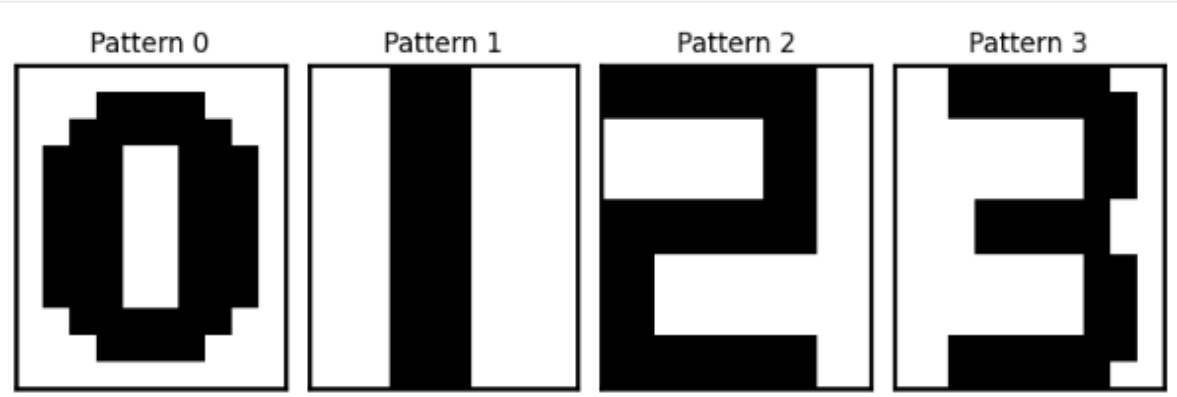
MODEL AND PERFORMANCES

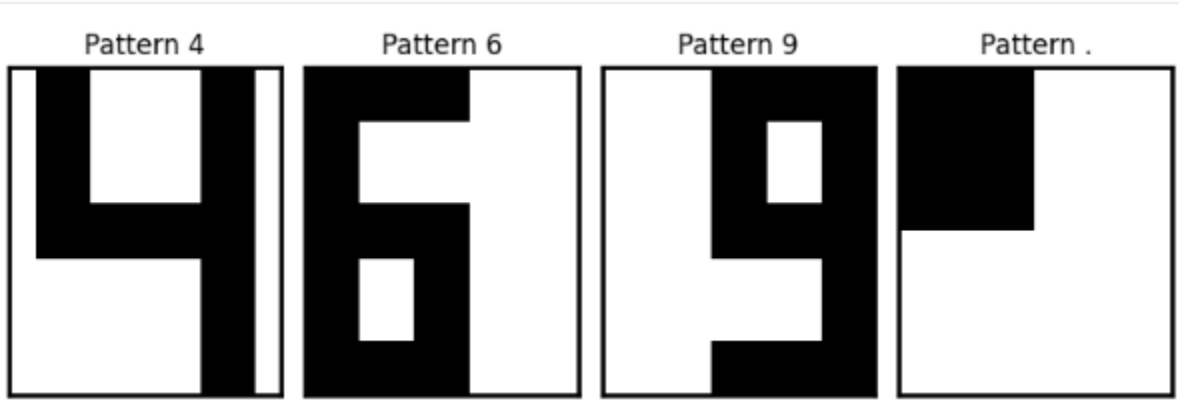
In this section, I will explain the steps I followed to design and test the Hopfield network. The goal of this project was to create an associator that can memorize seven specific digit patterns ("0", "1", "2", "3", "4", "6", "9", and “.”) and recall them even when noise is added

Network Design

First, a Hopfield network was created to store eight patterns: the digits **0, 1, 2, 3, 4, 6, 9**, and the symbol **"."**. Each pattern was represented as a small binary image where each pixel corresponds to a neuron in the network. The pixel values were converted into 120x1 state vector Using bipolar values, where **+1 represents an active pixel and −1 represents an inactive pixel**. Each pattern was then reshaped into a vector so it could be used as an input to the network.

The network was fully connected, meaning that each neuron was connected to all other neurons. However, there were **no self-connections**. A 120x120 weight matrix was generated using Hebbian learning, with self-connections disabled .





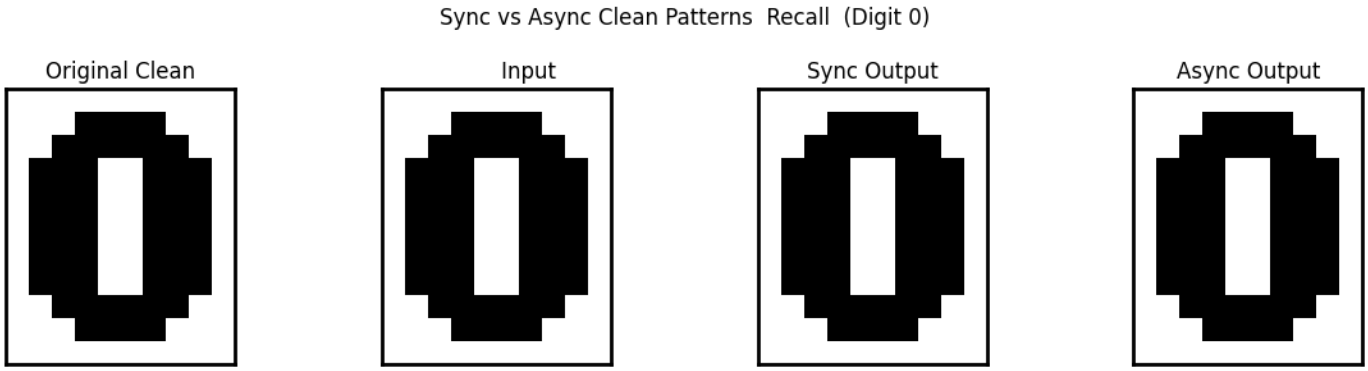
Training the Network

The network was trained using the **Hebbian learning rule**. All selected digit patterns were stored in the network by computing the weight matrix based on the correlation between neuron activations in the stored patterns. After calculating the weights, the network was able to act as an associative memory that can recall stored patterns.

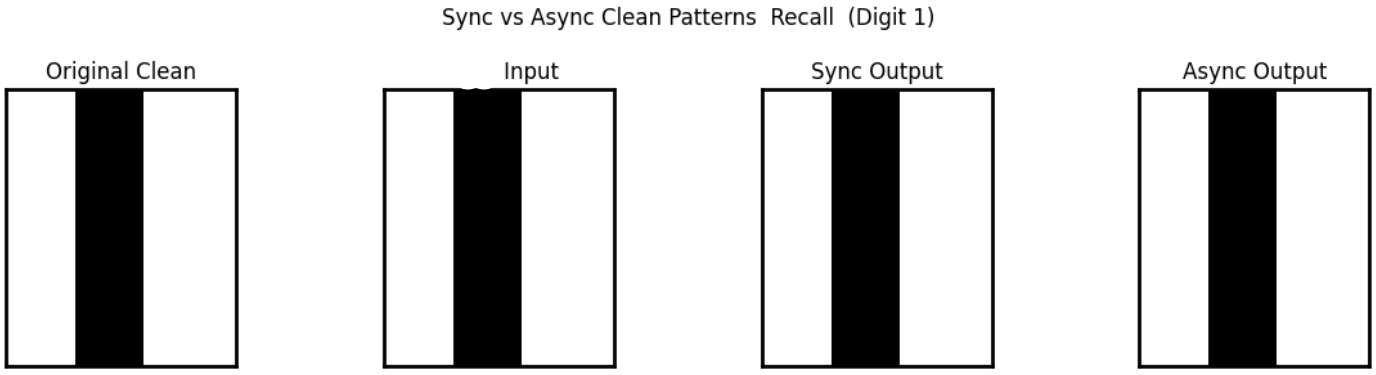
Testing with Clean Patterns (recall mode)

After training, the recall ability of the network was first tested using the **original clean patterns**. Each stored pattern was given as an input to the network, and the neuron states were updated until the network reached a stable state. The final output was then compared with the original stored pattern to check whether the network recalled the pattern correctly.

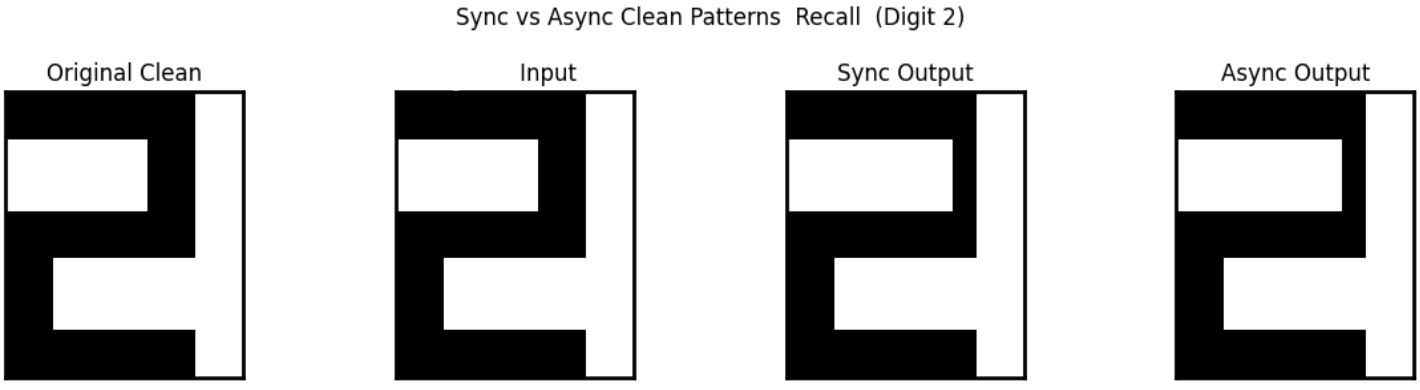
- **Sync Update:** Digit **0** recalled correctly**? True**
- **Async Update:** Digit **0** recalled correctly**? True**

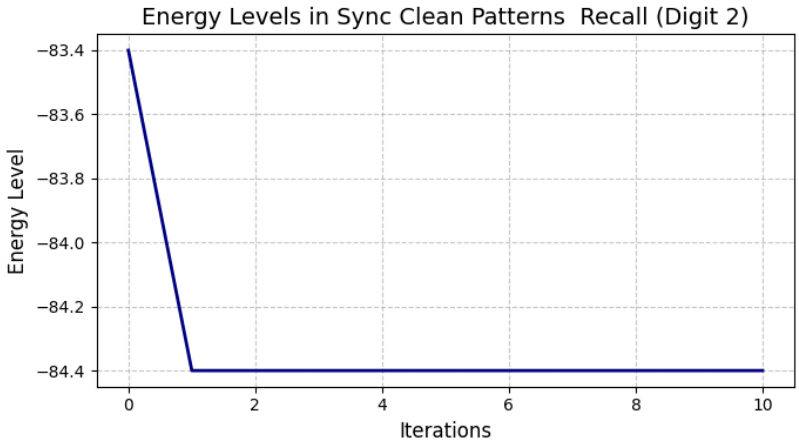


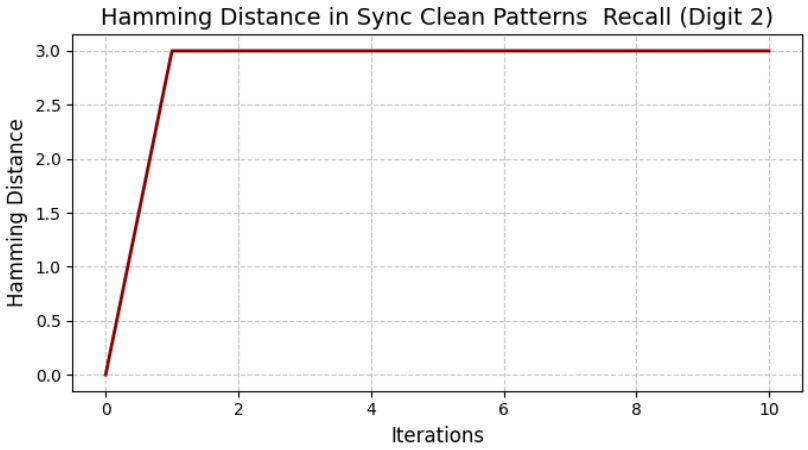
- **Sync Update:** Digit **1** recalled correctly**? True**
- **Async Update:** Digit **1** recalled correctly**? True**

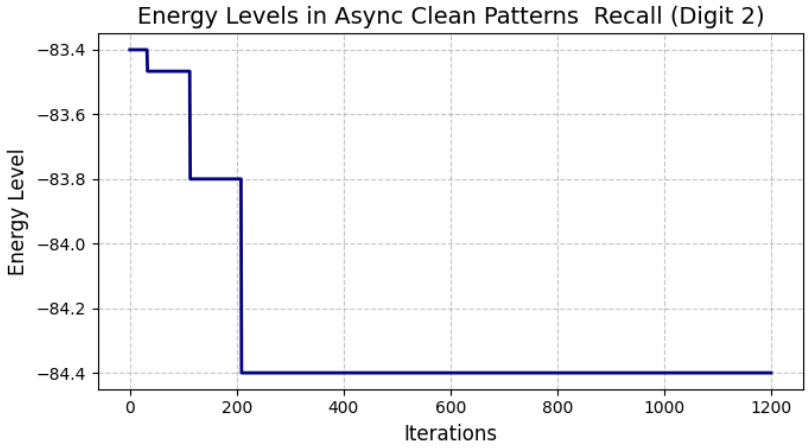


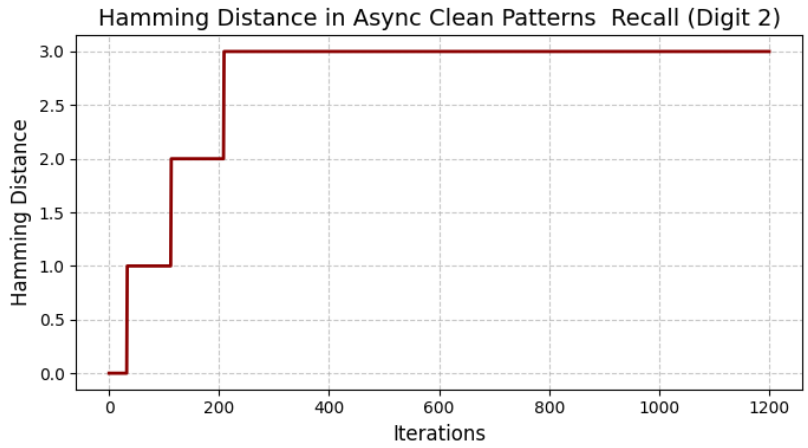
- **Sync Update:** Digit **2** recalled correctly**? False**
- **Async Update:** Digit **2** recalled correctly**? False**



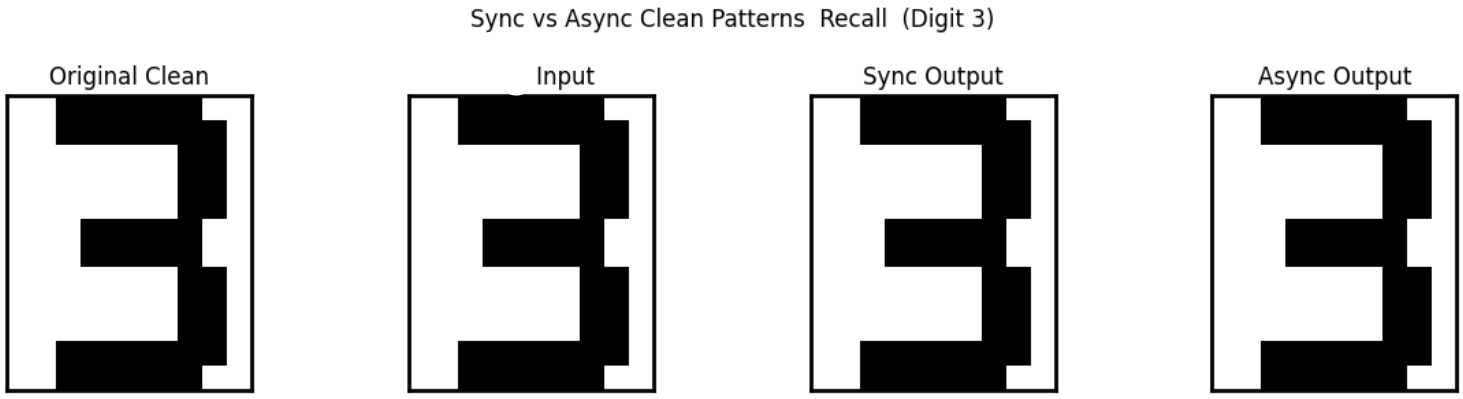




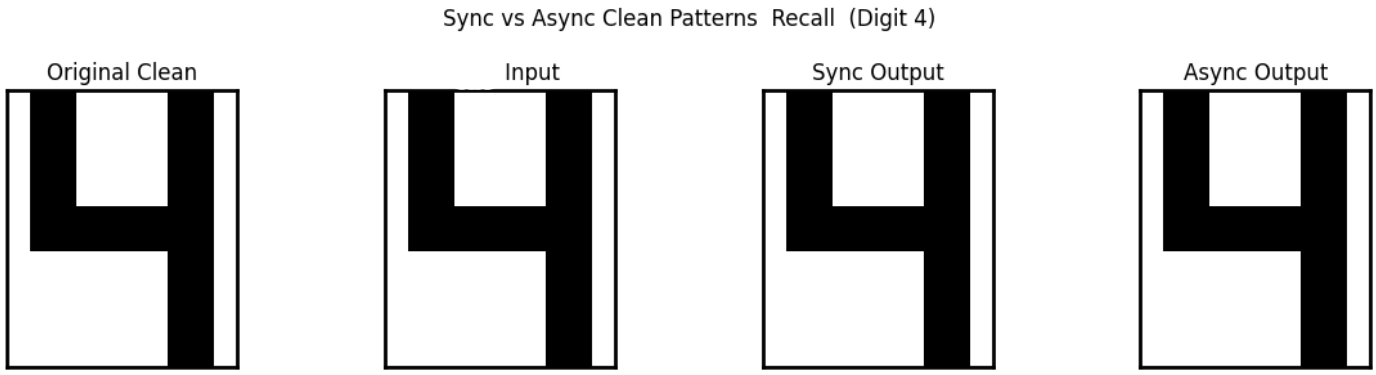




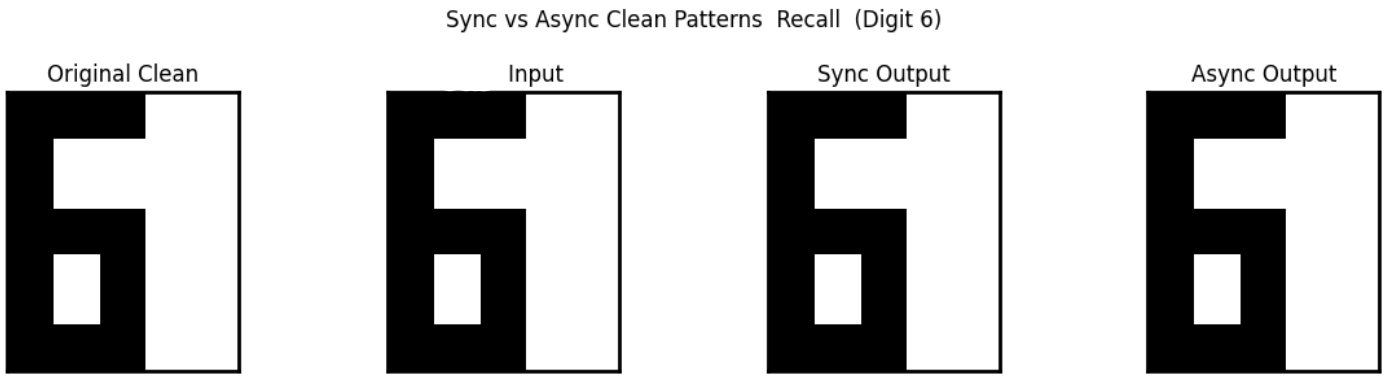
- **Sync Update:** Digit **3** recalled correctly**? True**
- **Async Update:** Digit **3** recalled correctly**? True**



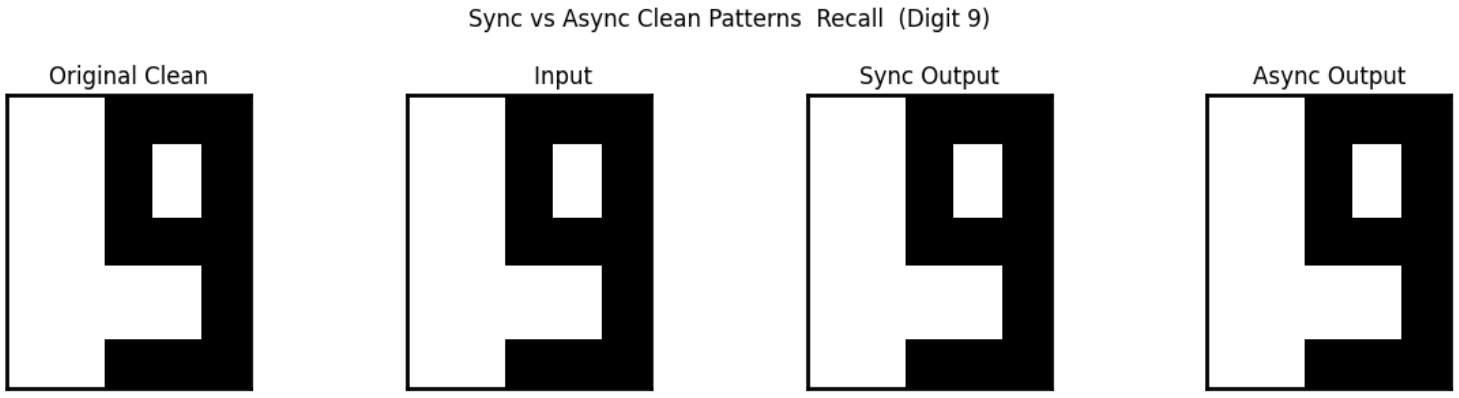
- **Sync Update:** Digit **4** recalled correctly**? True**
- **Async Update:** Digit **4** recalled correctly**? True**



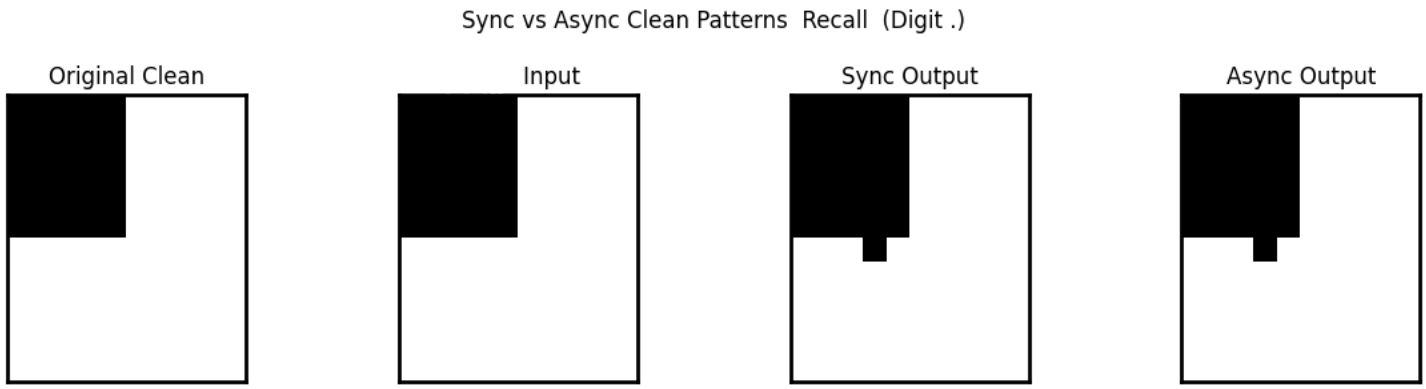
- **Sync Update:** Digit **6** recalled correctly**? True**
- **Async Update:** Digit **6** recalled correctly**? True**

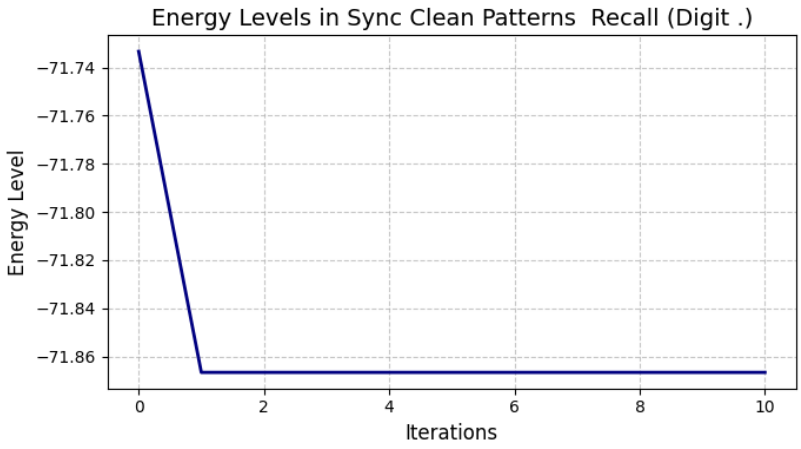


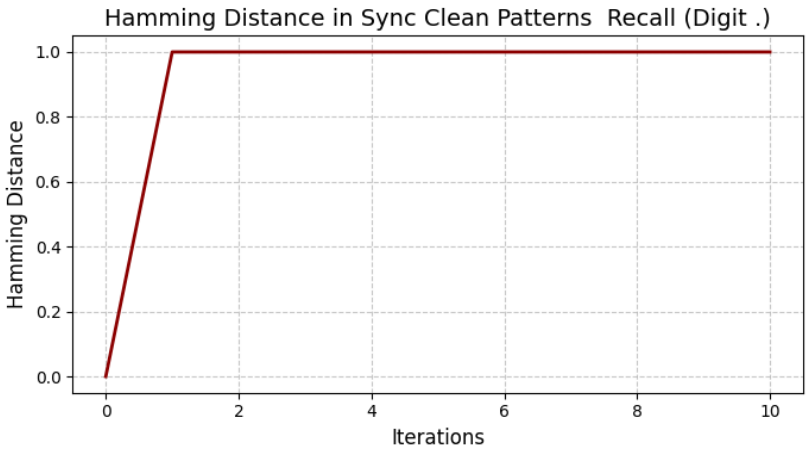
- **Sync Update:** Digit **9** recalled correctly**? True**
- **Async Update:** Digit **9** recalled correctly**? True**

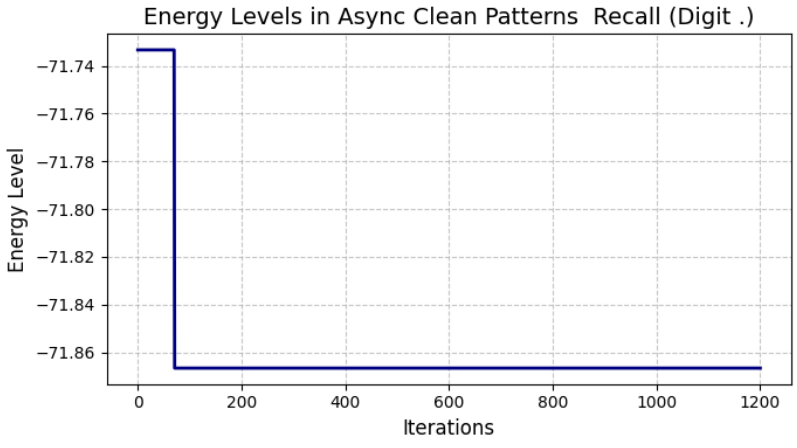


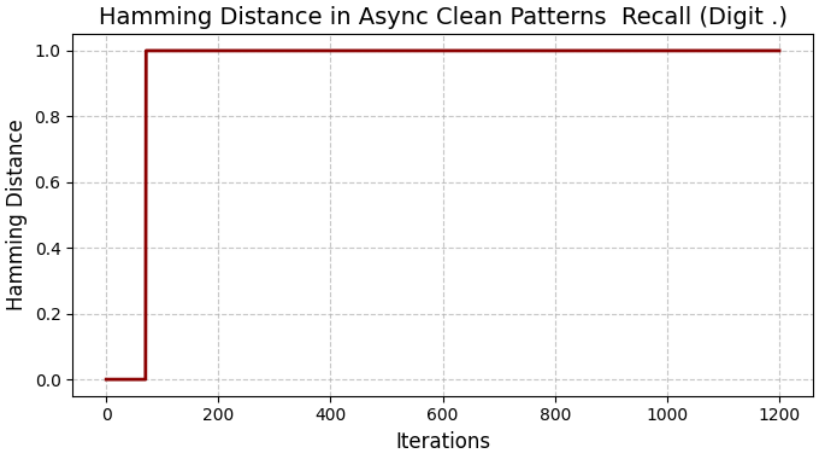
- **Sync Update:** Digit **.** recalled correctly**? False**
- **Async Update:** Digit **.** recalled correctly**? False**

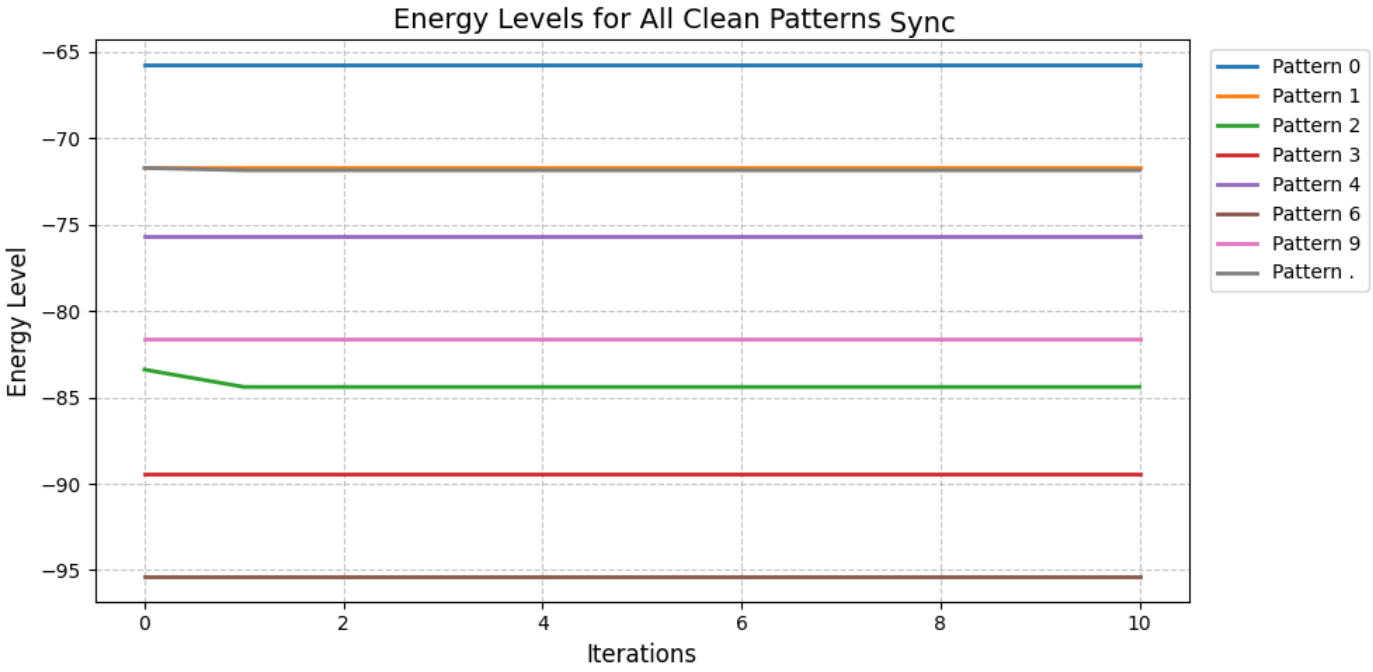


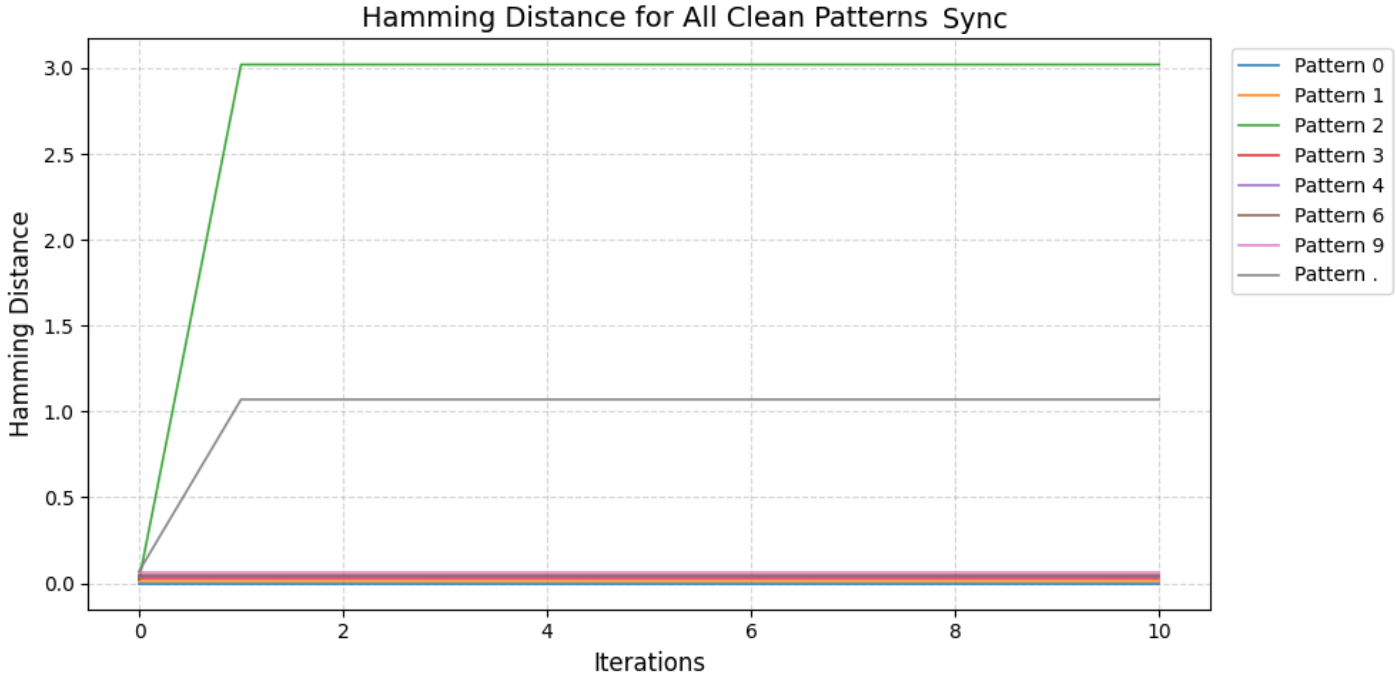


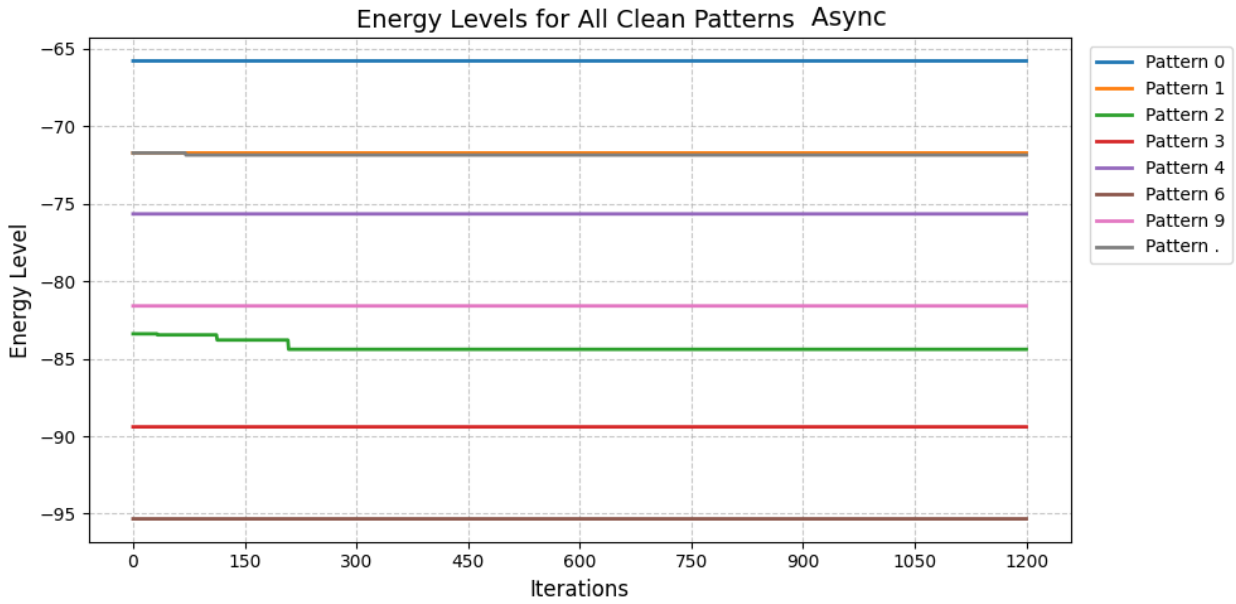


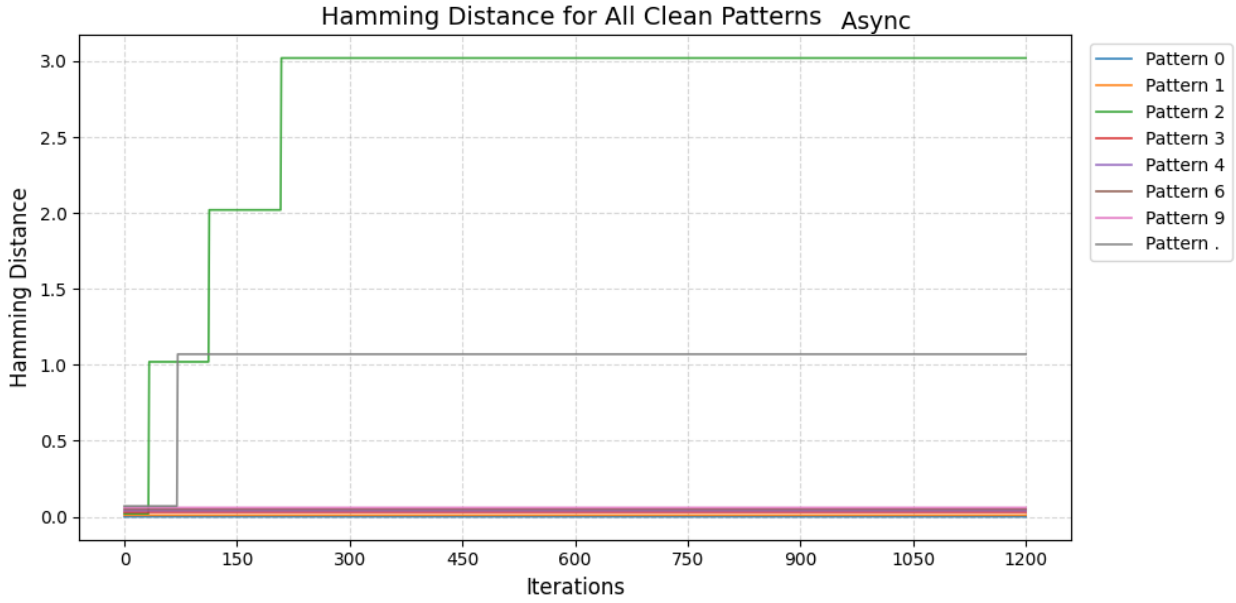





Energy Levels for All Clean Patterns Sync
Energy Level
Iterations
−65
−70
−75
−80
−85
−90
−95
0
2
4
6
8
10
Pattern 0
Pattern 1
Pattern 2
Pattern 3
Pattern 4
Pattern 6
Pattern 9
Pattern .


Hamming Distance for All Clean Patterns Sync
Hamming Distance
Iterations
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
2
4
6
8
10
Pattern 0
Pattern 1
Pattern 2
Pattern 3
Pattern 4
Pattern 6
Pattern 9
Pattern .





The results show that the Hopfield network correctly recalled most of the clean patterns. The patterns **0, 1, 3, 4, 6, and 9** were recognized perfectly by the network. In the plots, the **Hamming distance is 0**, which means the output pattern is exactly the same as the stored pattern. Also, the **energy stays constant**, showing that the network is already in a stable state.

However, the network did not keep pattern **"2"** and pattern **"."** completely unchanged. The results show a **Hamming distance of 3 pixels for pattern "2"** and **1 pixel for pattern "."**. The energy values also decreased slightly. This happens because of that Some of the digit patterns are very similar

and share many pixels. Because of this similarity, the network becomes confused and changes a few pixels while trying to minimize its energy.

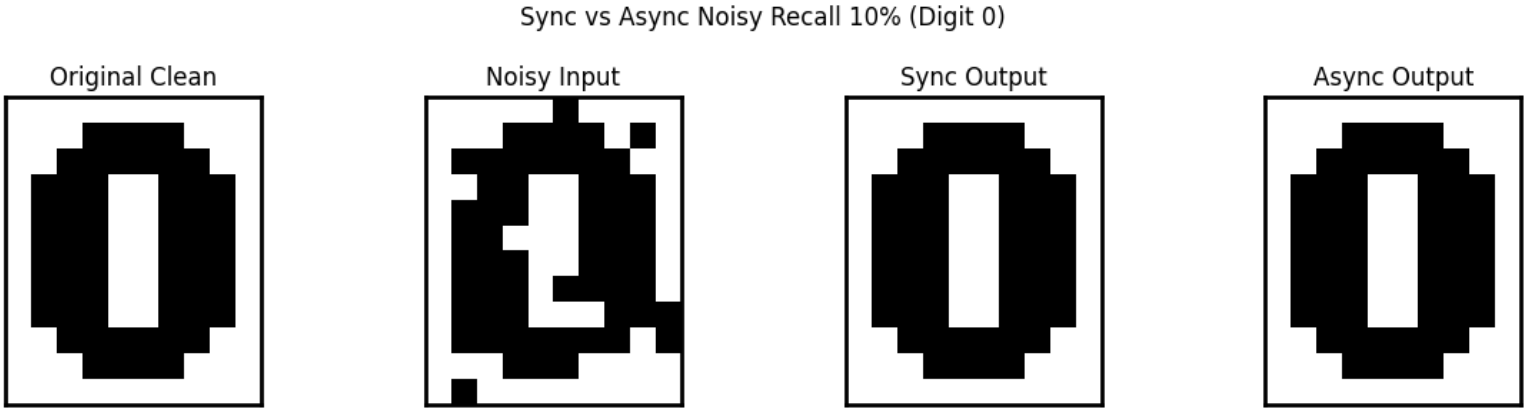
Testing with Noisy Patterns

To evaluate the robustness of the network, the stored patterns were corrupted by adding 2 different amount of noises. In this experiment, **10% and 25 of the pixels in each pattern were randomly reversed**. This means that some pixel values were changed from +1 to −1 or from −1 to +1. These noisy patterns were then used as inputs to the network to see whether the Hopfield network could reconstruct the correct stored patterns.

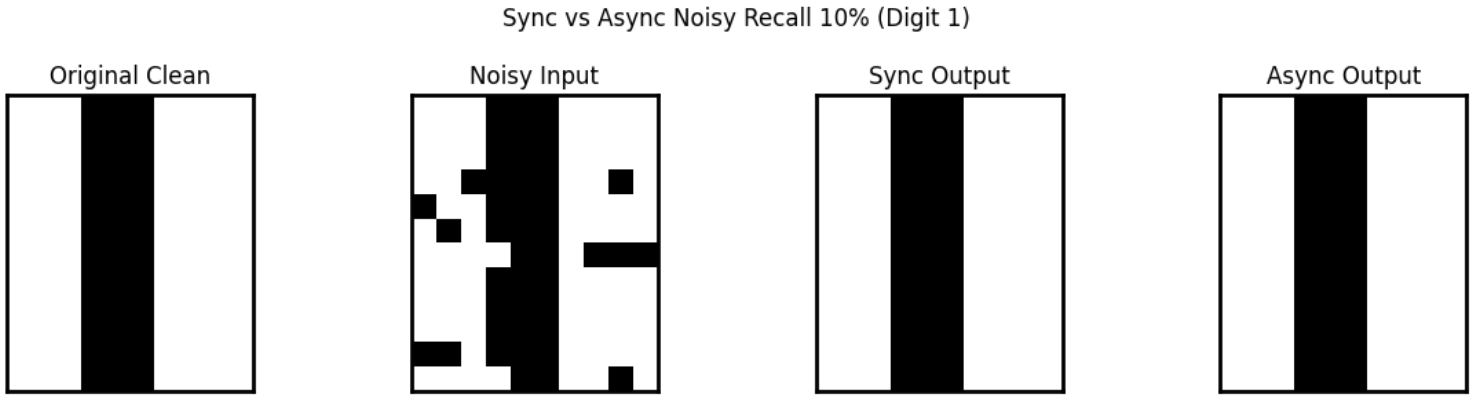
Synchronous and Asynchronous update:

In these methods, first all neurons in the network update their states at the same time during each iteration (Synchronous), and then only one neuron updates its state at a time, The neurons are updated sequentially until the network reaches a stable state (Asynchronous).

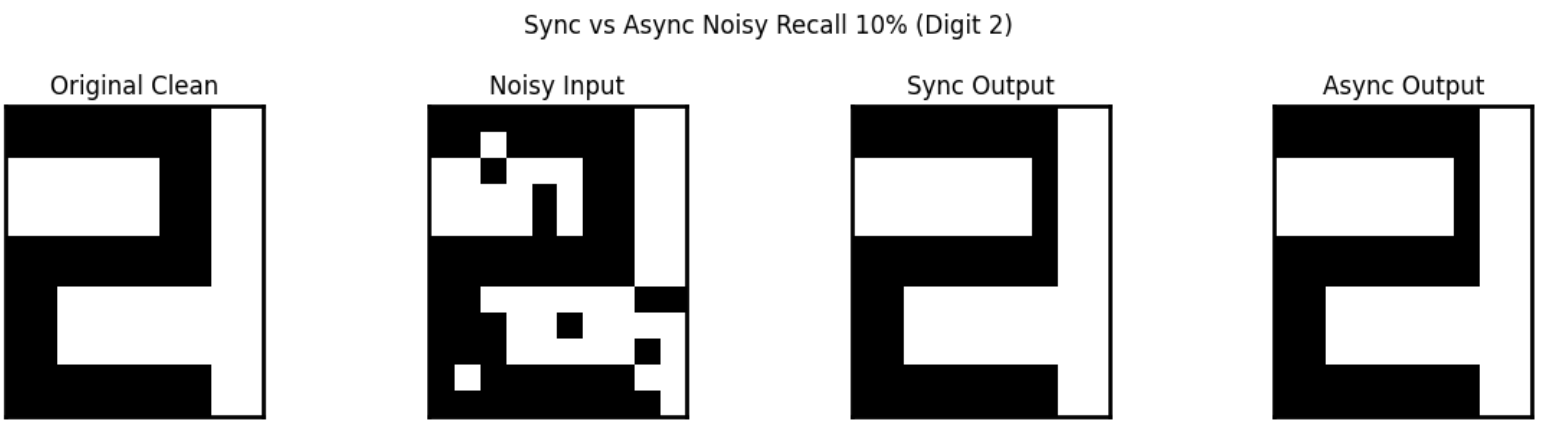
- **With 10% of noise:**
    - **Sync Update:** Digit **0** recalled correctly**? True**
    - **Async Update:** Digit **0** recalled correctly**? True**

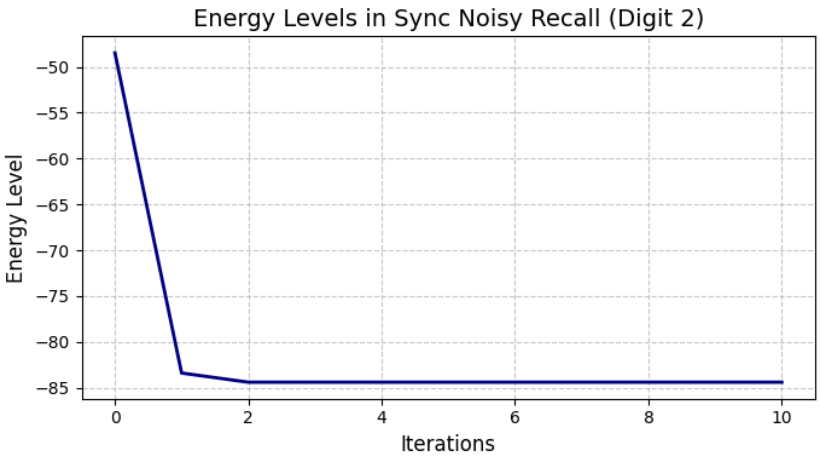


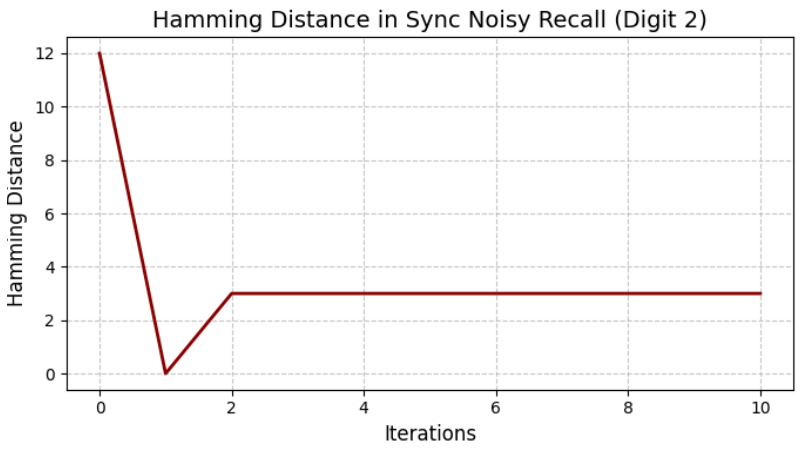
- **Sync Update:** Digit **1** recalled correctly**? True**
- **Async Update:** Digit **1** recalled correctly**? True**

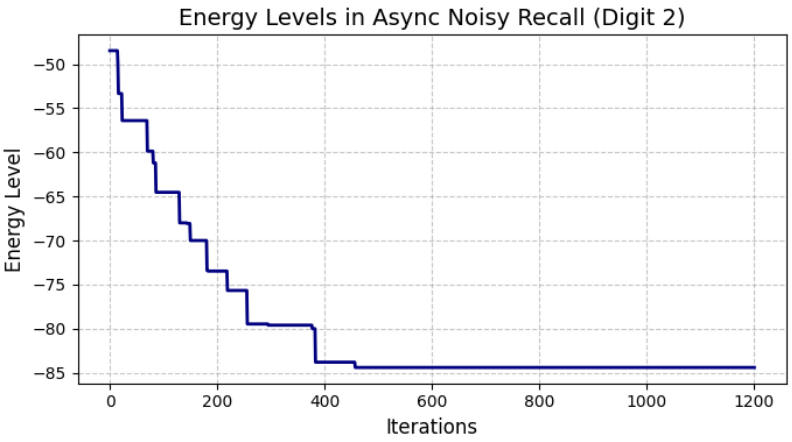


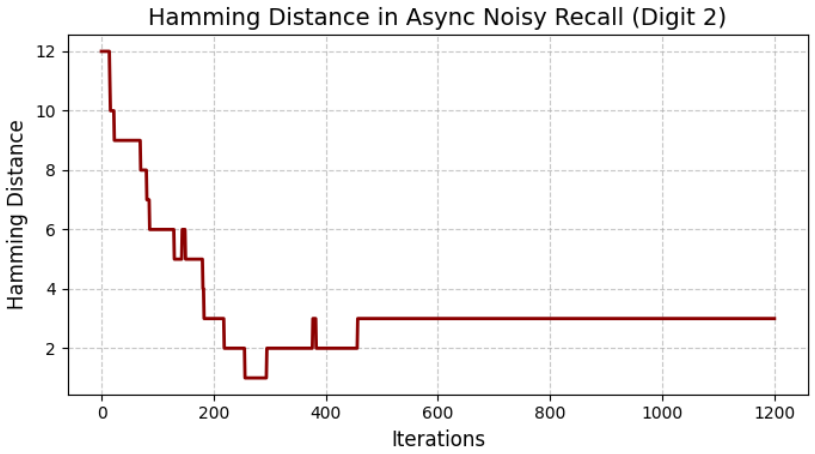
- **Sync Update:** Digit **2** recalled correctly**? False**
- **Async Update:** Digit **2** recalled correctly**? False**



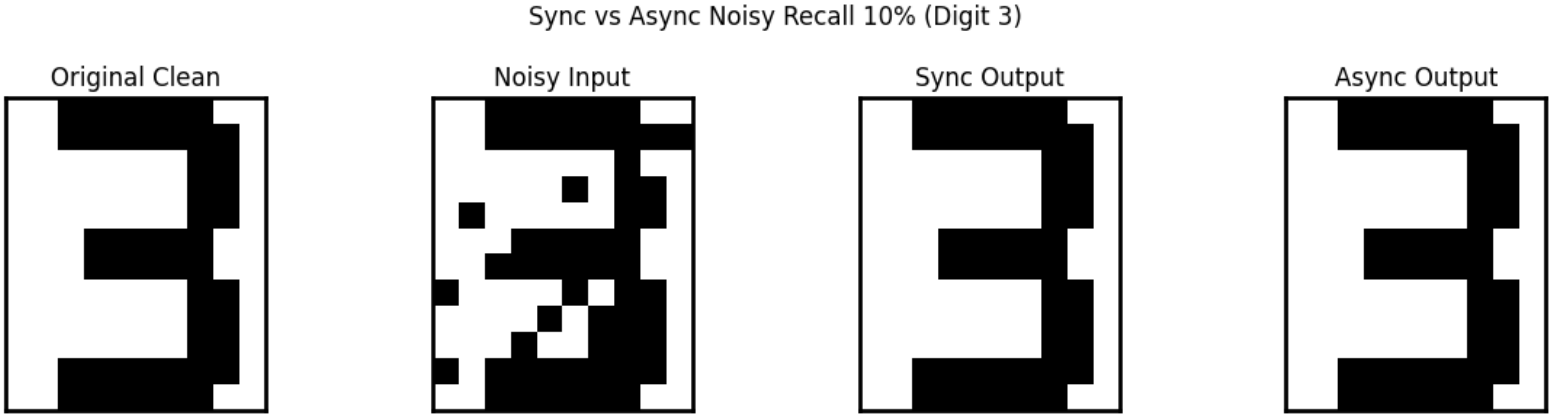




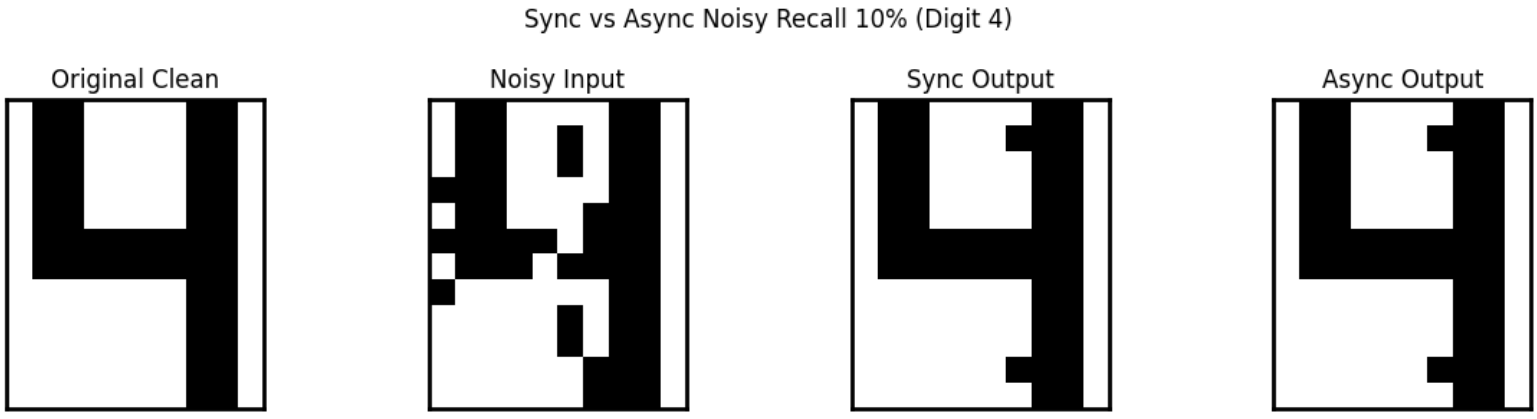


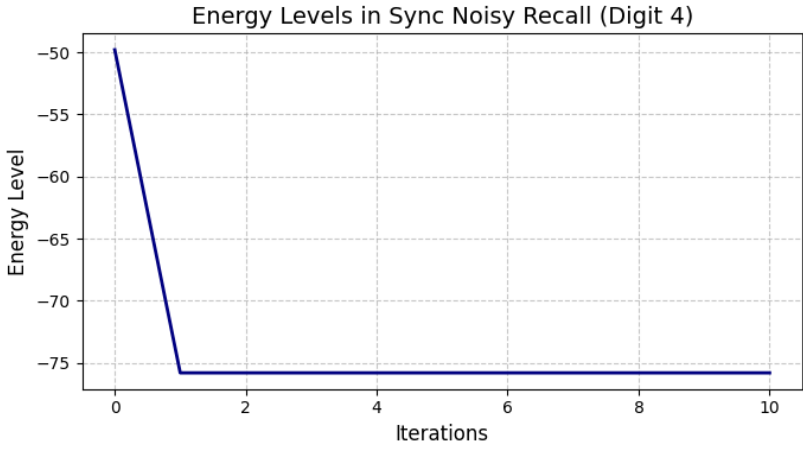


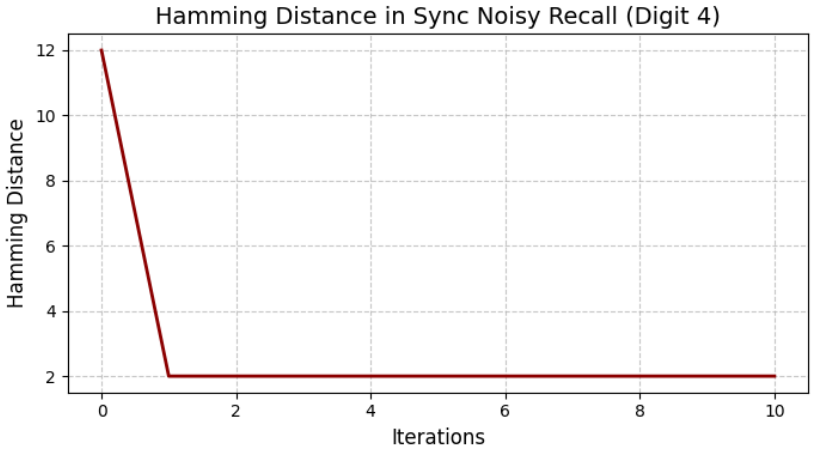
- **Sync Update:** Digit **3** recalled correctly**? True**
- **Async Update:** Digit **3** recalled correctly**? True**

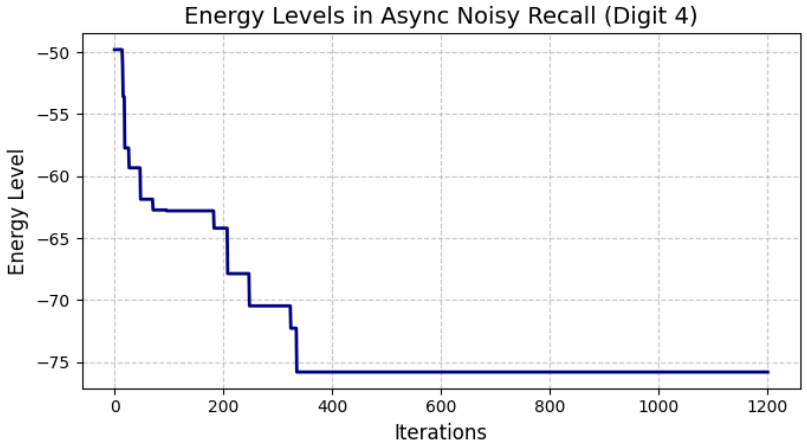


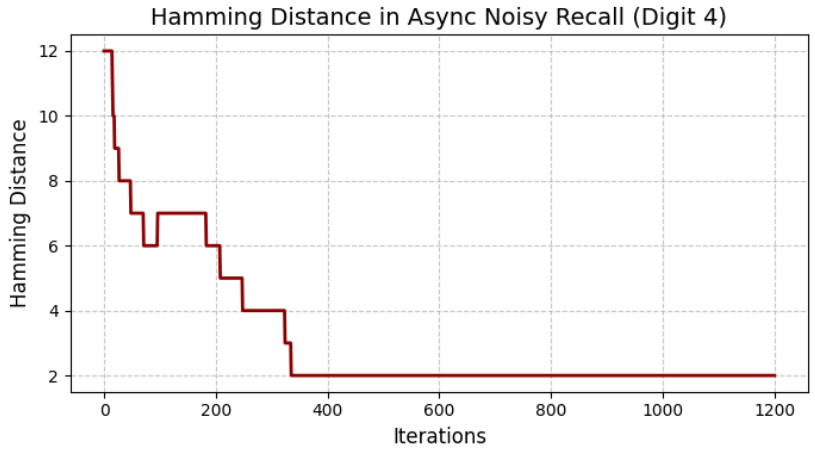
- **Sync Update:** Digit **4** recalled correctly**? False**
- **Async Update:** Digit **4** recalled correctly**? False**



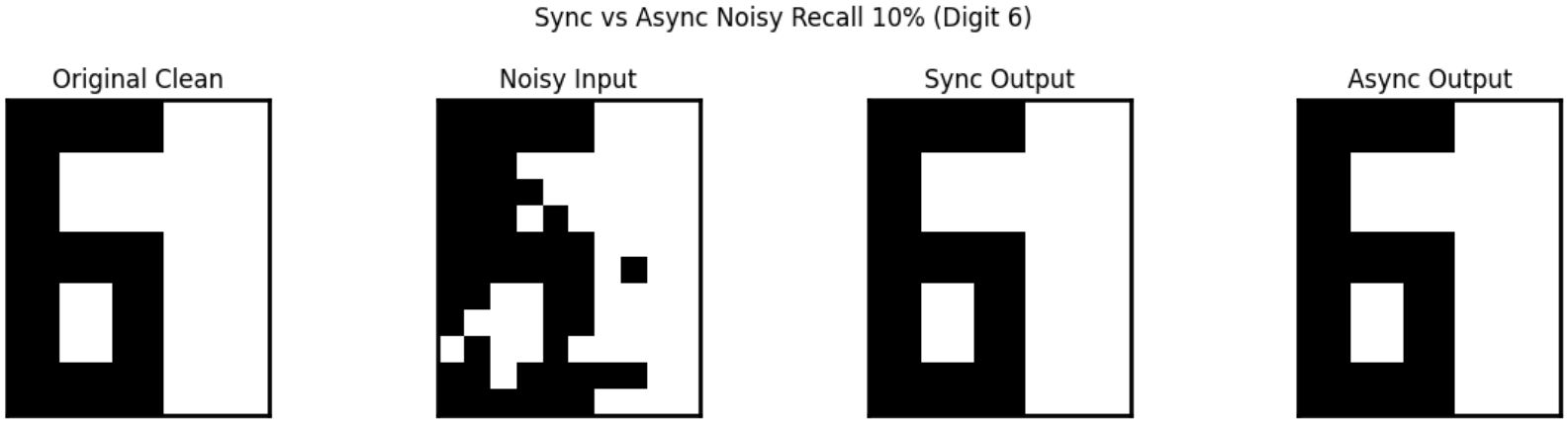




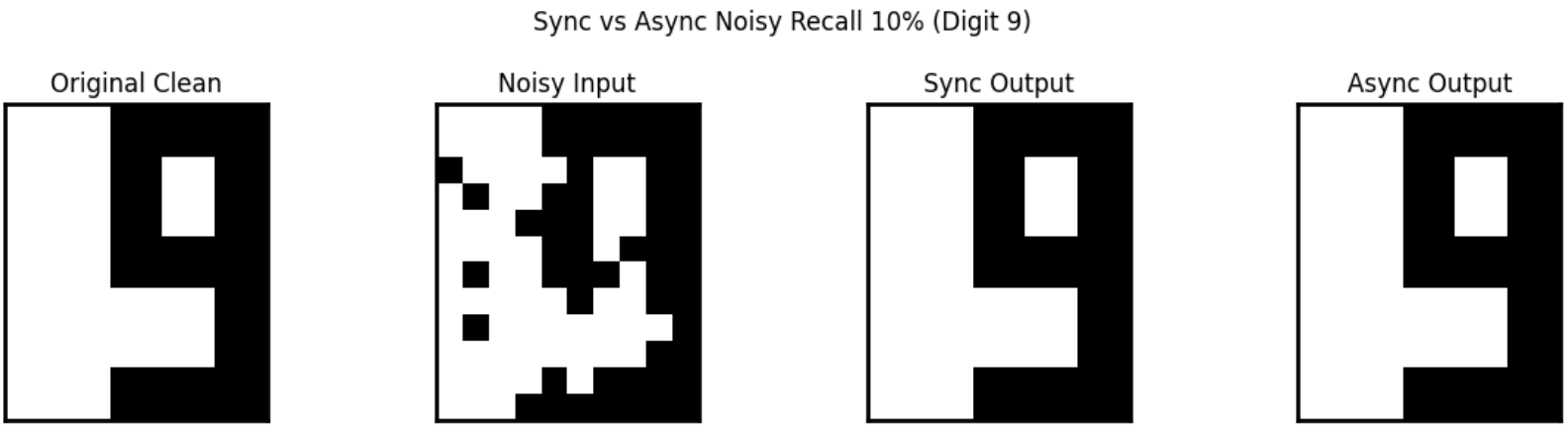




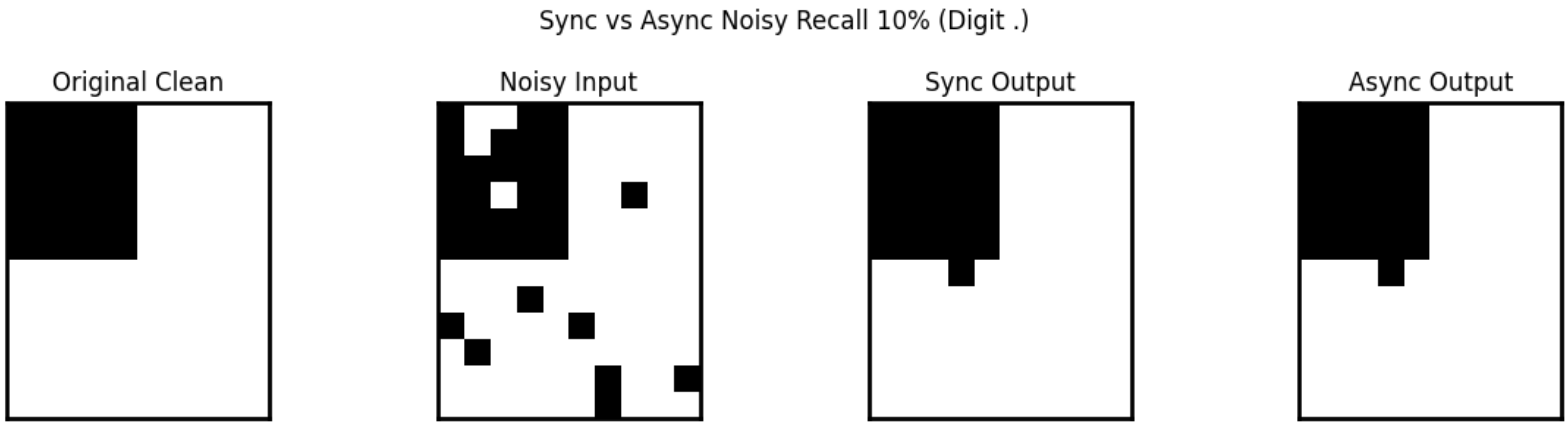
- **Sync Update:** Digit **6** recalled correctly**? True**
- **Async Update:** Digit **6** recalled correctly**? True**

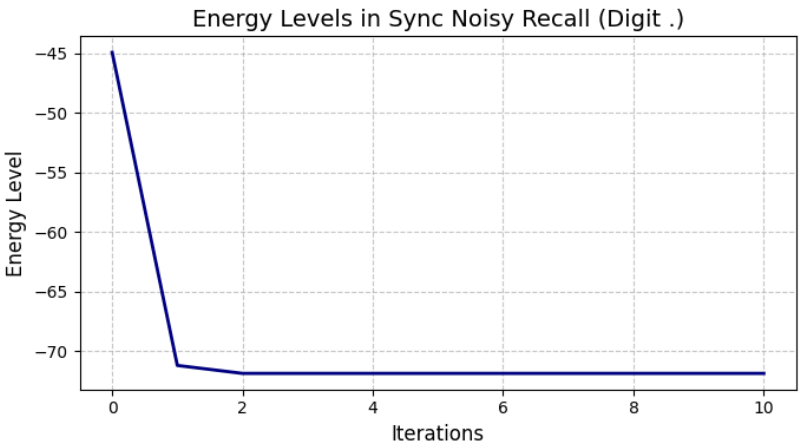


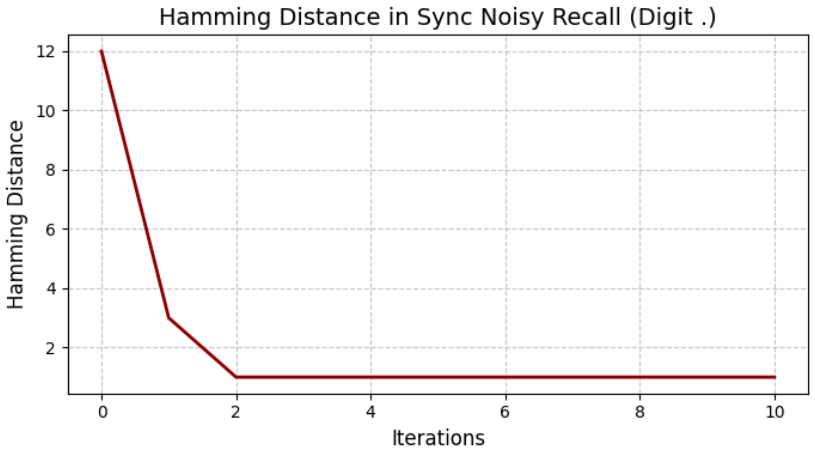
- **Sync Update:** Digit **9** recalled correctly**? True**
- **Async Update:** Digit **9** recalled correctly**? True**

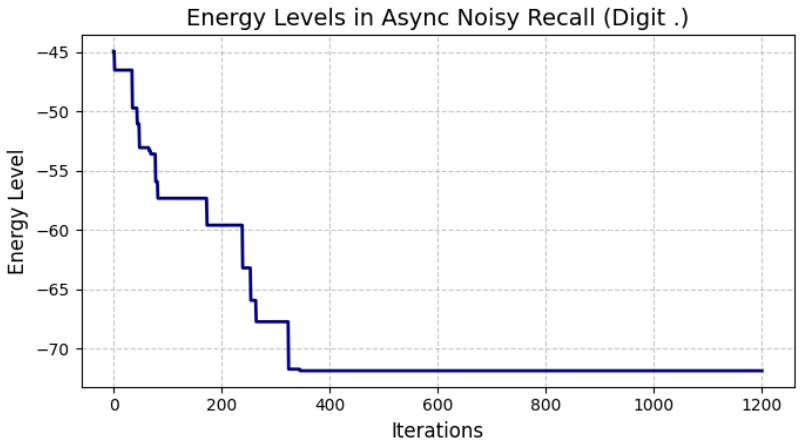


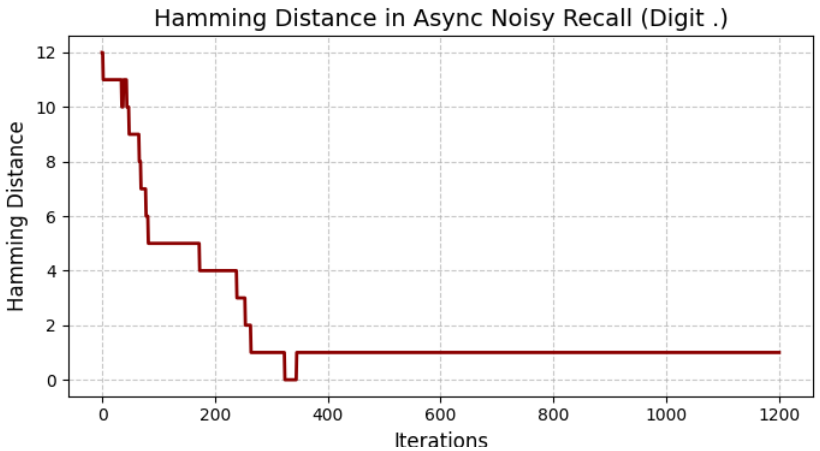
- **Sync Update:** Digit **.** recalled correctly**? False**
- **Async Update:** Digit **.** recalled correctly**? False**

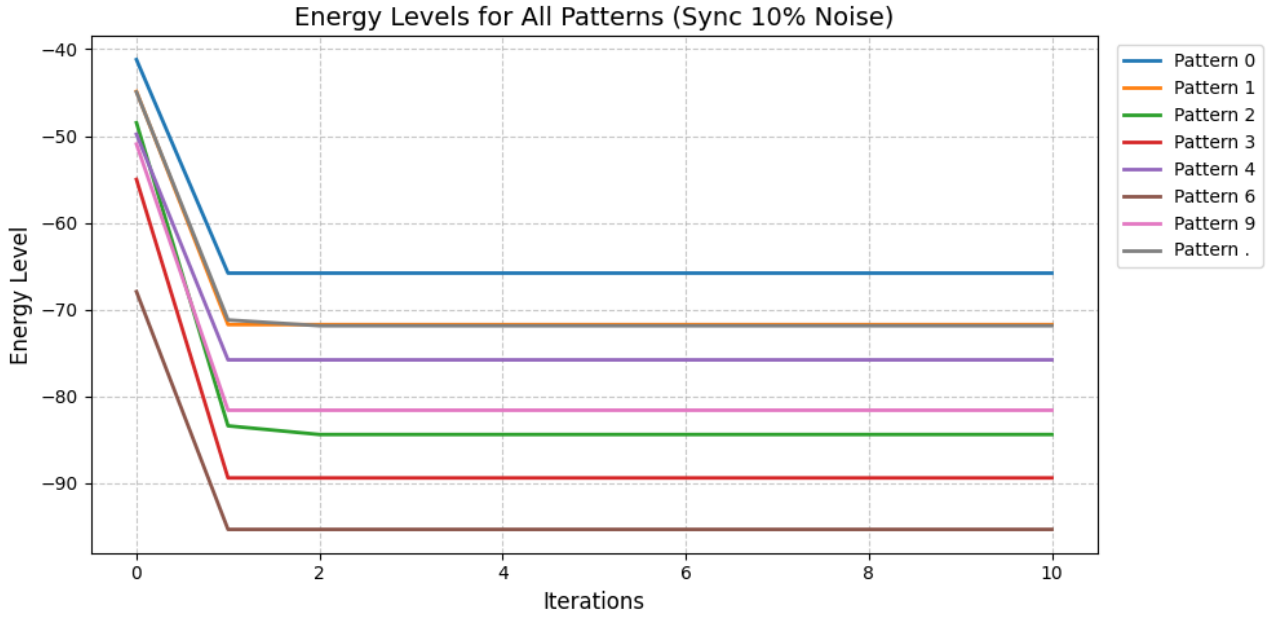


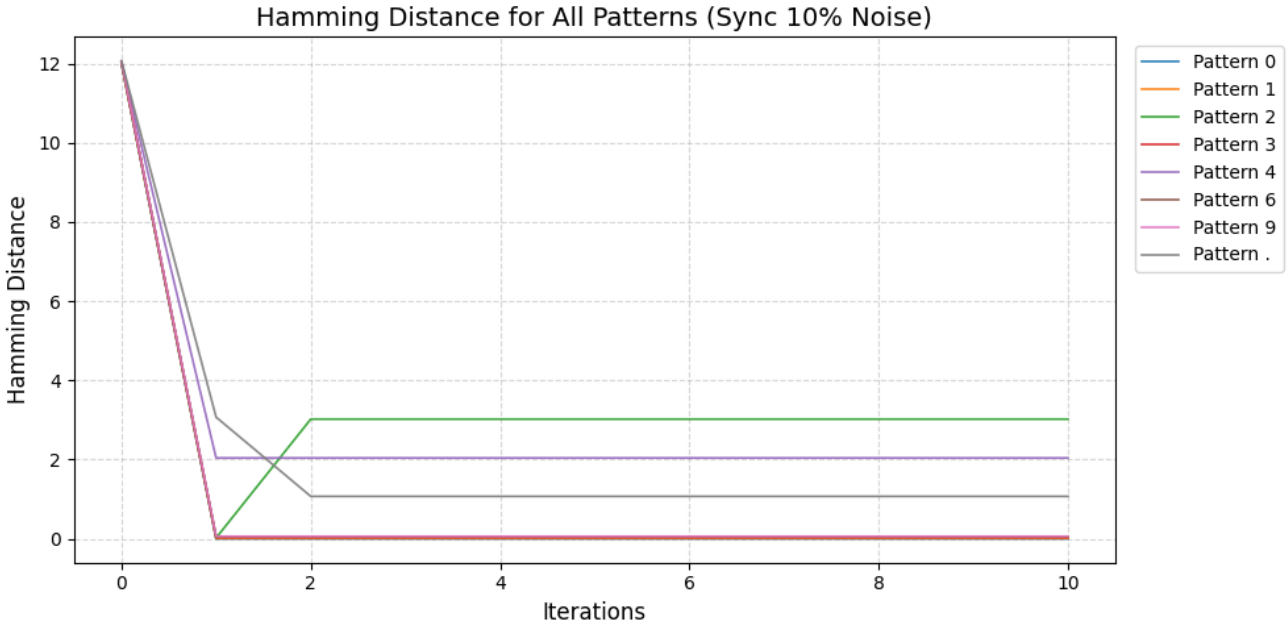





Energy Levels in Async Noisy Recall (Digit .)
Energy Level
Iterations
−45
−50
−55
−60
−65
−70
0
200
400
600
800
1000
1200


Hamming Distance in Async Noisy Recall (Digit .)
Hamming Distance
Iterations
12
10
8
6
4
2
0
0
200
400
600
800
1000
1200


Energy Levels for All Patterns (Sync 10% Noise)
Energy Level
Iterations
−40
−50
−60
−70
−80
−90
0
2
4
6
8
10
Pattern 0
Pattern 1
Pattern 2
Pattern 3
Pattern 4
Pattern 6
Pattern 9
Pattern .

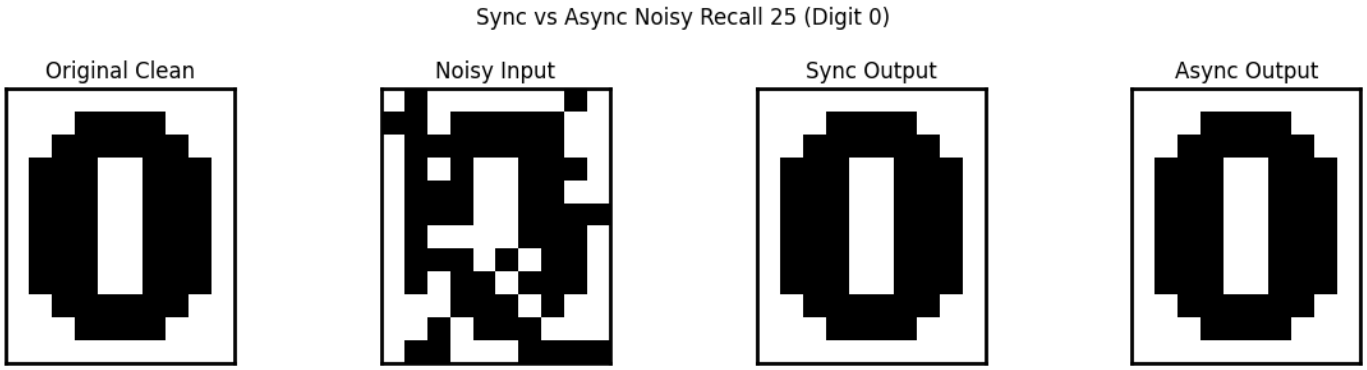

Hamming Distance for All Patterns (Sync 10% Noise)
Hamming Distance
Iterations
12
10
8
6
4
2
0
0
2
4
6
8
10
Pattern 0
Pattern 1
Pattern 2
Pattern 3
Pattern 4
Pattern 6
Pattern 9
Pattern .

In this test, we added 10% noise (12 flipped pixels) to all patterns. The Hopfield network was able to **correctly recover** most digits such as **0, 1, 3, 6, and 9**, and their Hamming distance became 0.

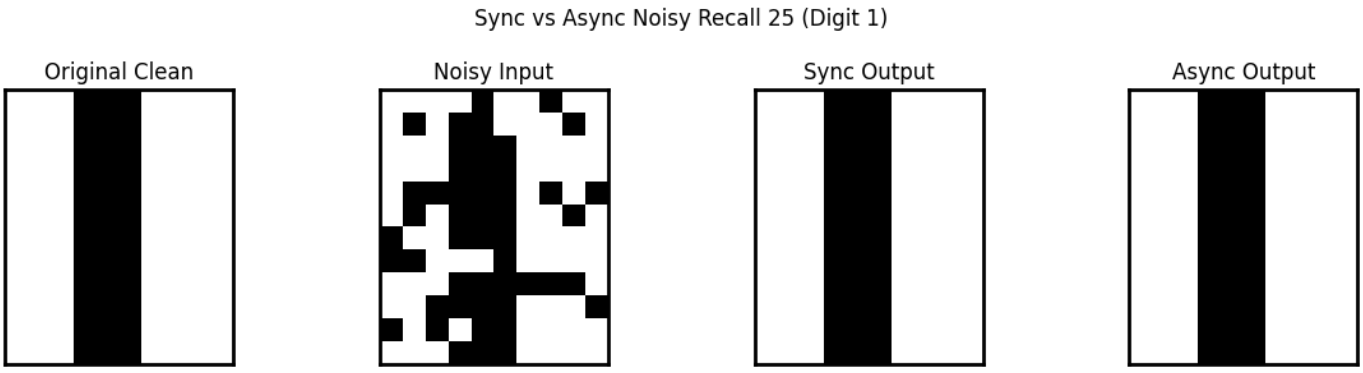
However, some similar patterns like "**2", "4", and "."** were **not recovered perfectly**, so only the plots for these failed patterns are shown. For these patterns, the energy still decreases, which means the network is trying to remove the noise. But instead of reaching the correct pattern, it gets stuck in a false memory.

A clear example is pattern "2". In synchronous mode, the network first fixes all 12 noisy pixels and the Hamming distance becomes 0. But the clean pattern "2" is not fully stable because it is similar to other digits. So the network changes 3 pixels again to reach a lower energy state.

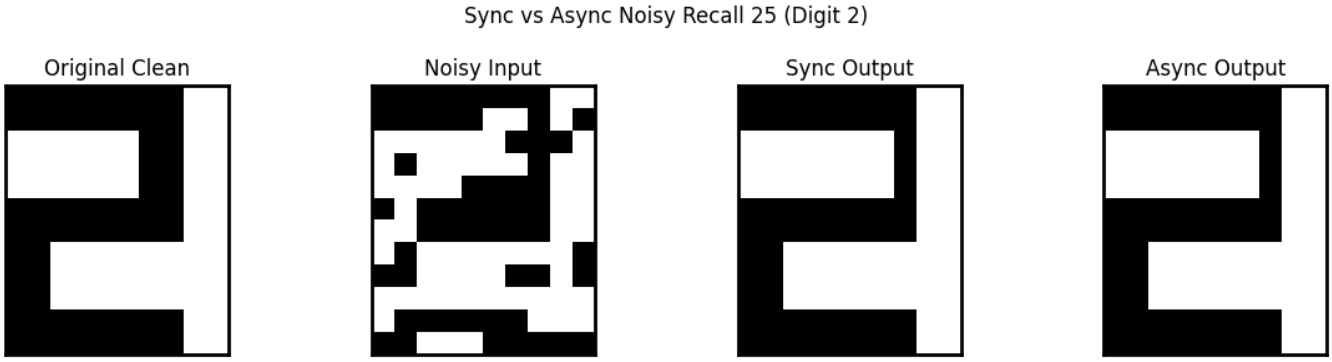
- **With 25 pixels noises**
    - **Sync Update**: Digit **0** recalled correctly? **True**
    - **Async Upda**te: Digit **0** recalled correctly? **True**

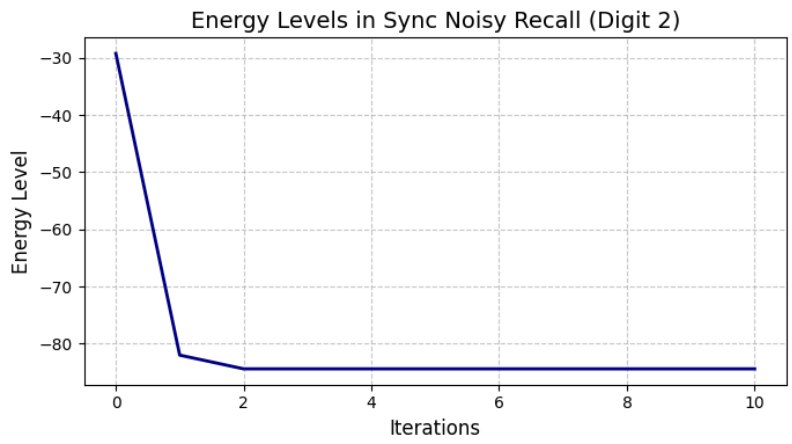


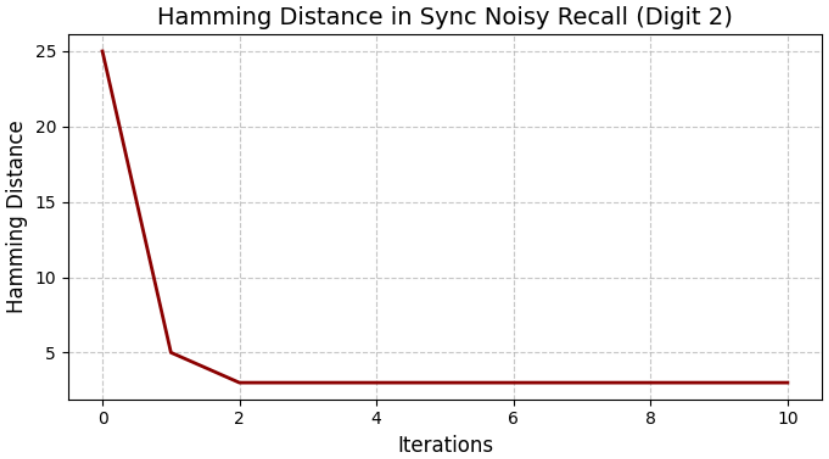
    - **Sync Update:** Digit **1** recalled correctly**? True**
    - **Async Update:** Digit **1** recalled correctly**? True**

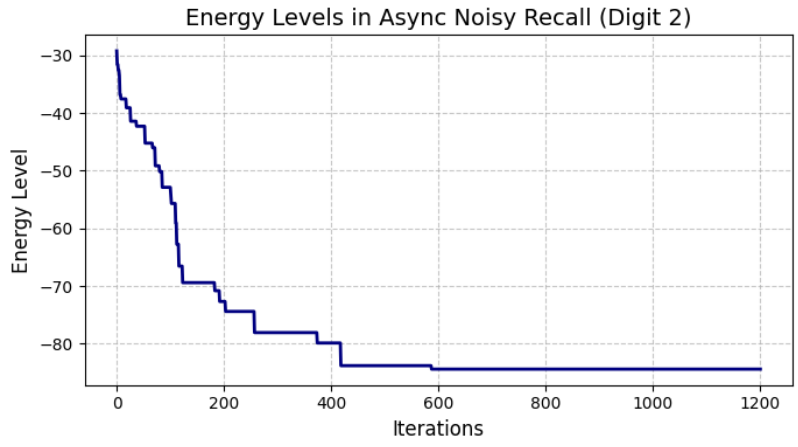


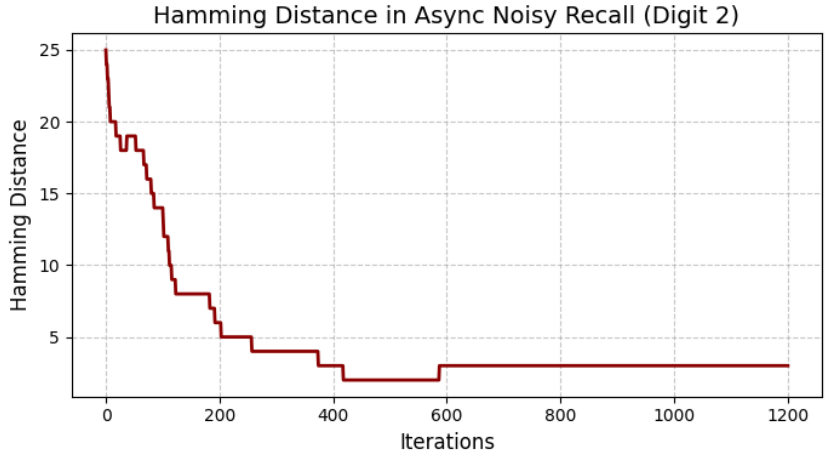
- **Sync Update:** Digit **2** recalled correctly**? False**
- **Async Update:** Digit **2** recalled correctly**? False**



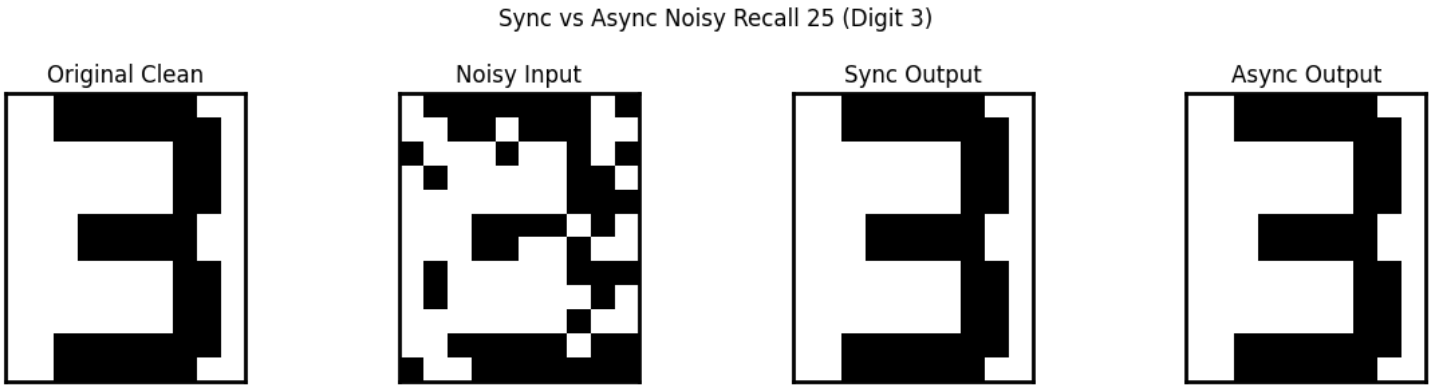




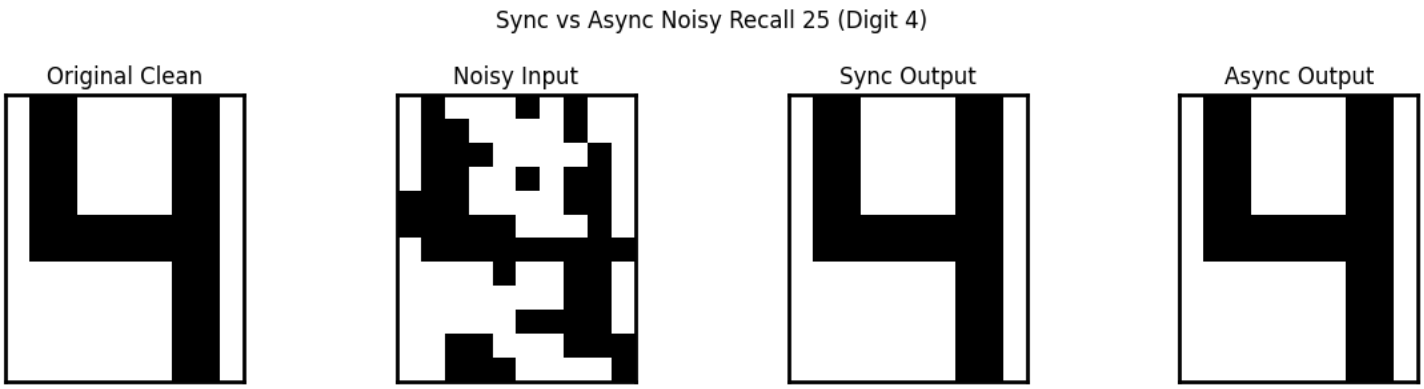




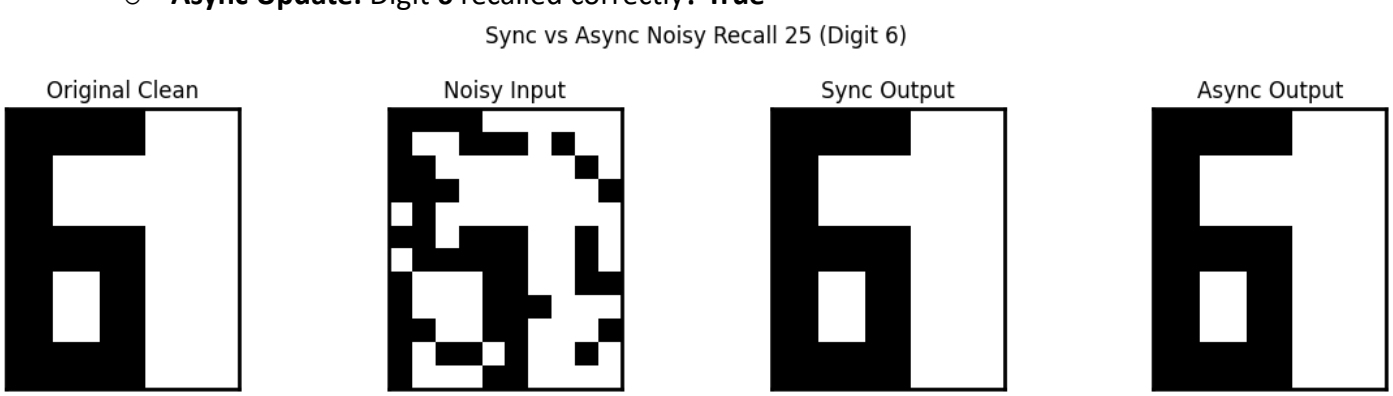
- **Sync Update:** Digit **3** recalled correctly**? True**
- **Async Update:** Digit **3** recalled correctly**? True**



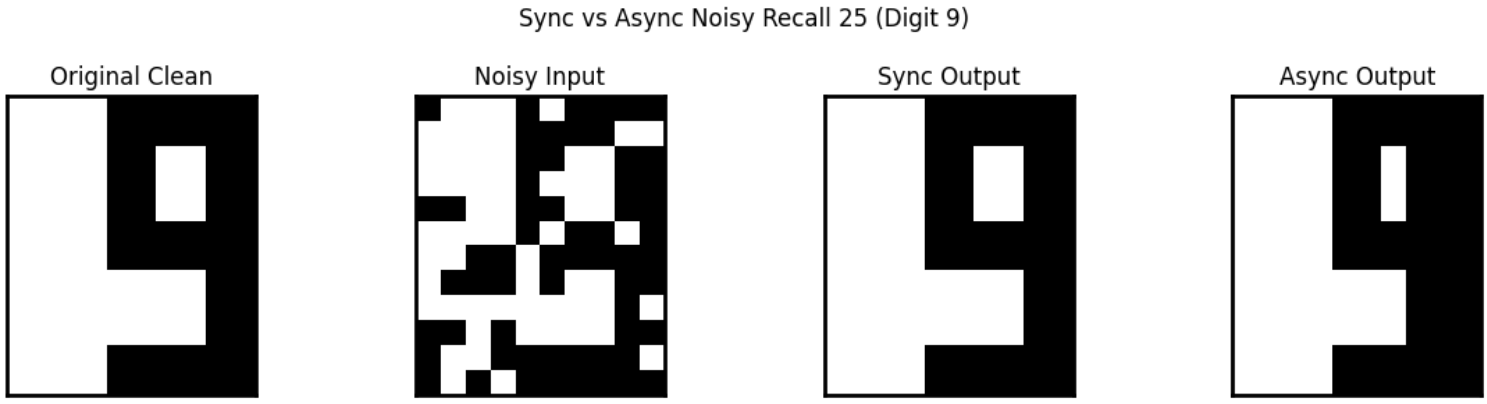
- **Sync Update:** Digit **4** recalled correctly**? True**
- **Async Update:** Digit **4** recalled correctly**? True**



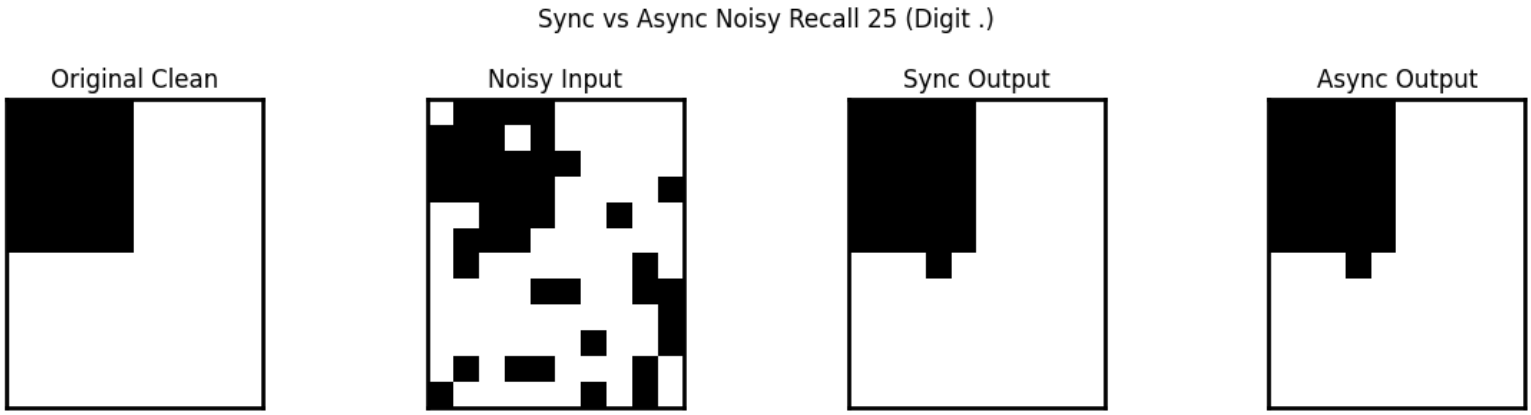
- **Sync Update:** Digit **6** recalled correctly**? True**
- **Async Update:** Digit **6** recalled correctly**? True**

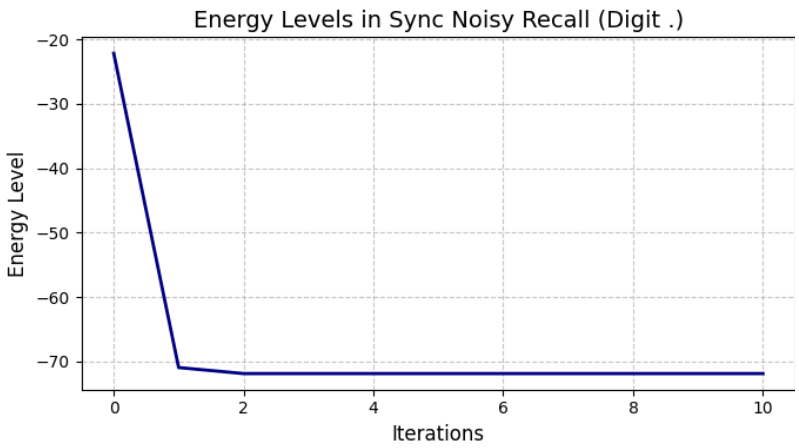


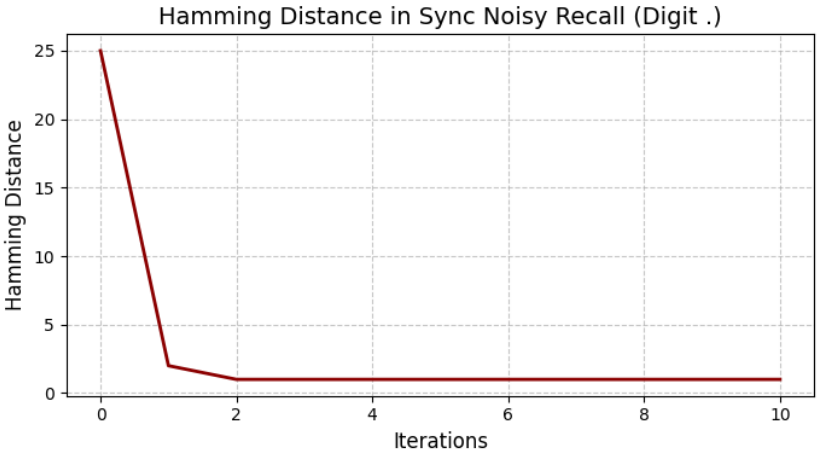
- **Sync Update:** Digit **9** recalled correctly**? True**
- **Async Update:** Digit **9** recalled correctly**? False**

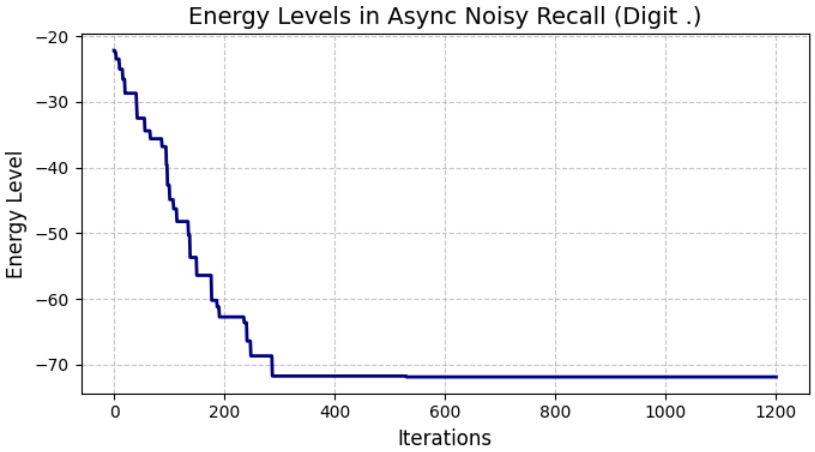


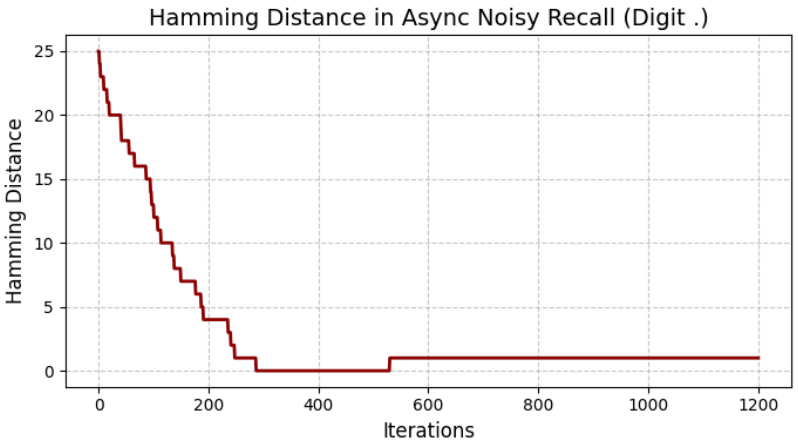
- **Sync Update: Digit .** recalled correctly**? False**
- **Async Update: Digit .** recalled correctly**? False**

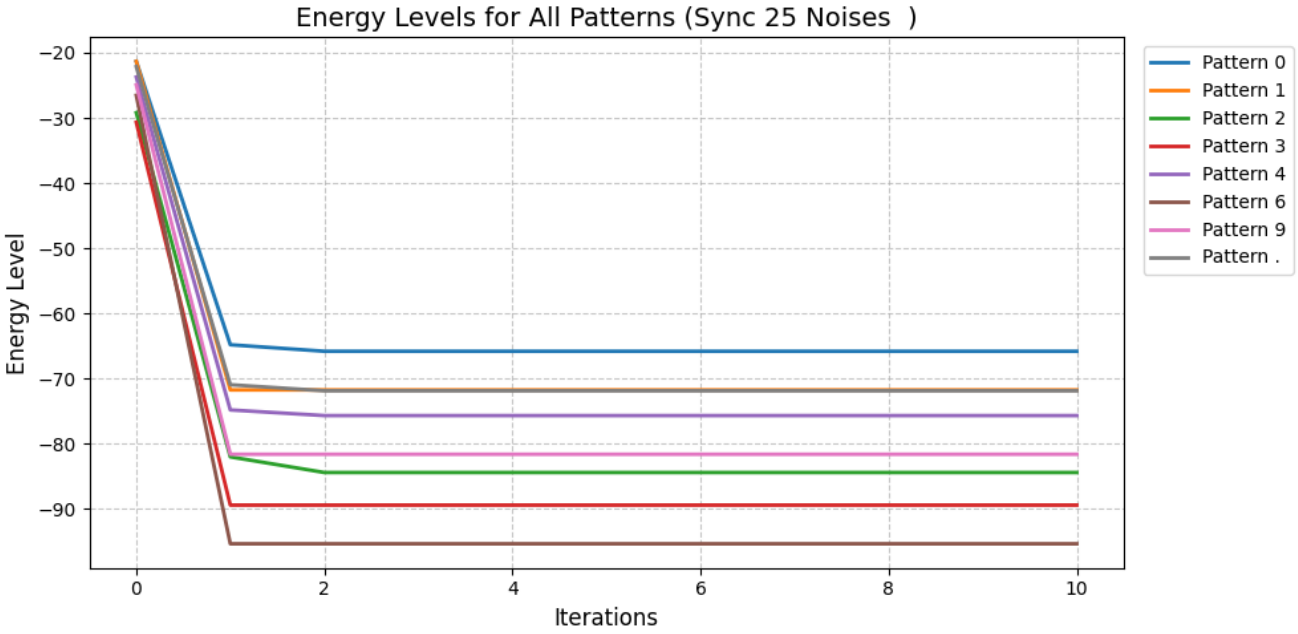


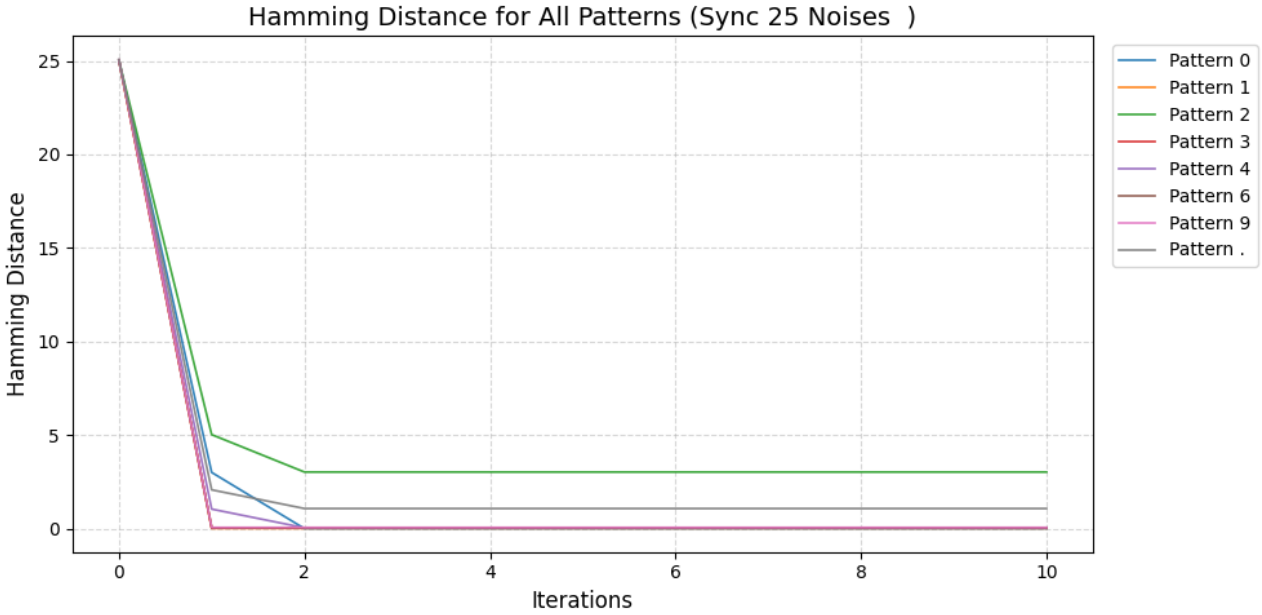


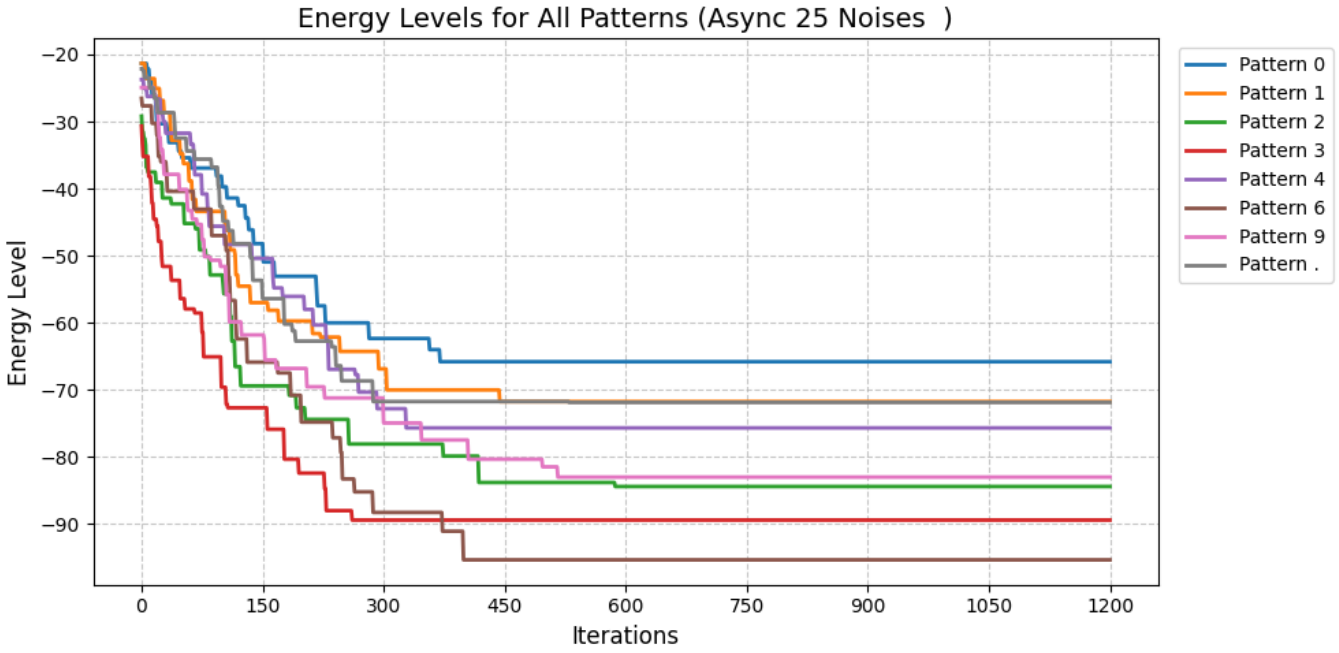


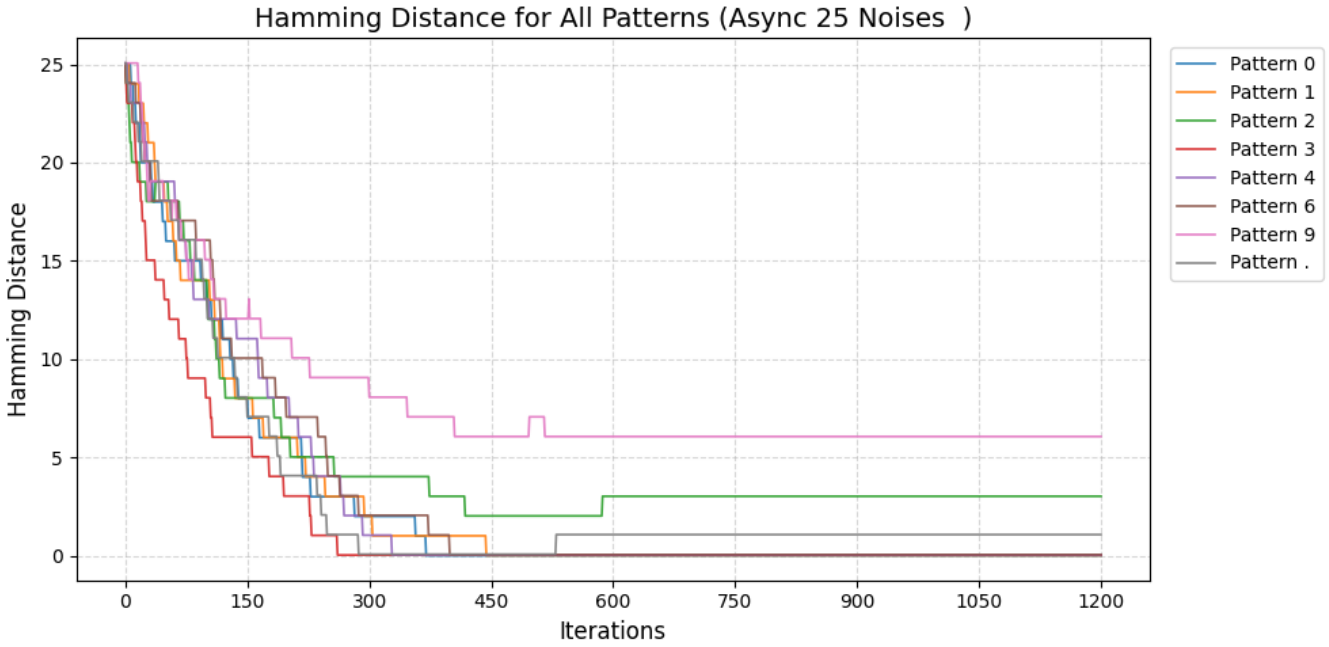

Energy Levels in Async Noisy Recall (Digit .)
Energy Level
Iterations


Hamming Distance in Async Noisy Recall (Digit .)
Hamming Distance
Iterations


Energy Levels for All Patterns (Sync 25 Noises)
Energy Level
Iterations
Pattern 0
Pattern 1
Pattern 2
Pattern 3
Pattern 4
Pattern 6
Pattern 9
Pattern .


Hamming Distance for All Patterns (Sync 25 Noises)
Hamming Distance
Iterations
Pattern 0
Pattern 1
Pattern 2
Pattern 3
Pattern 4
Pattern 6
Pattern 9
Pattern .





In this phase, the Hopfield network was tested by adding **25 bits of noise** to the stored patterns. For most digits, like other tests, the network successfully recovered the original pattern.

Both **synchronous and asynchronous** updates show that the **energy decreases**, which means the network is moving toward a stable state.

However, **pattern "." and digit "2"** did not recover perfectly and got stuck in a **wrong pattern**. In these cases, the Hamming distance became smaller, but **did not reach 0**, which shows a limitation of the network.